

Ministry of Education and Science of the Russian Federation
ST. PETERSBURG ELECTROTECHNICAL UNIVERSITY «LETI»

**V.B. VTOROV, K.V. IGNATEV,
M.M. KOPICHEV**

ADAPTIVE CONTROL IN MECHATRONICS AND ROBOTICS

Training manual for laboratory works

Saint Petersburg
2023

V.B. Vtorov, K.V. Ignatev, M.M. Kopichev. Training manual for laboratory works on the course "Adaptive Control in Mechatronics and Robotics". St. Petersburg, 2023.

The training manual for laboratory works on the course «Adaptive Control in Mechatronics and Robotics» (Master's Degree Program: «Automation and Mechatronics», field of study: 27.04.04 – «Control in Technical Systems») contains brief theoretical information and examples of solving problems on basic content of the course.

1. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS WITH PARAMETRIC ADAPTATION FOR FIRST-ORDER PLANTS

Goal of the work: mastering the skills of adaptive control systems investigation, studying the adaptive control effectiveness in presence of plant parameters changing, and researching the adaptive control systems ability to stabilize the plant.

Theoretical background. Direct adaptation system with parametric adjustment. Consider linear time-invariant plant of n order with single input in following form

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{b}u, \quad (1.1)$$

where $\mathbf{x} \in R^n$ – plant state vector (1.1), $u = u_a + u_0$ – scalar control action, including u_0 – reference input, u_a – adaptive control action, $\mathbf{b} \in R^n$ – plant input matrix, $\mathbf{A} \in R^{n \times n}$ – plant state matrix.

Assume that $\mathbf{A}, \mathbf{b}$ are not known, but we know their average (nominal) values $\mathbf{A}_0, \mathbf{b}_0$. In this case, we can use a well-known adaptive control law of the following form

$$u_a = \mathbf{k}_A^T \mathbf{x} + k_b u_0, \quad (1.2)$$

where column vector $\mathbf{k}_A$ and scalar k_b – are adjustable parameters of the adaptive control law (1.2).

Desired adaptive system dynamics is given by a reference model of the following form

$$\dot{\mathbf{x}}_M = \mathbf{A}_M \mathbf{x}_M + \mathbf{b}_M u_0, \quad (1.3)$$

where $\mathbf{A}_M, \mathbf{b}_M$ – $n \times n$ and $n \times 1$ -dimensional constant matrices, $\mathbf{A}_M$ – square matrix, whose all eigenvalues are located in the left half-plane, $\mathbf{x}_M \in R^n$ – reference model state vector, u_0 – arbitrary continuous bounded function of time.

Differential equations of $\mathbf{k}_A, k_b$ adjustment have following form

$$\begin{cases} \dot{\mathbf{k}}_A^T = -\mathbf{b}_M^T \mathbf{P} \mathbf{e} \mathbf{x}^T \Gamma_A, \\ \dot{k}_b = -\gamma_b \mathbf{b}_M^T \mathbf{P} \mathbf{e} u_0, \end{cases} \quad (1.4)$$

where $\mathbf{e} = \mathbf{x} - \mathbf{x}_M$ – n -dimensional error vector, characterizing the deviation of the plant state from the reference model state, Γ_A – symmetric positive definite matrix, γ_b – positive adjustment algorithm gain, $\mathbf{P}$ – symmetric positive definite matrix satisfying the Lyapunov equation of the following form

$$\mathbf{A}_M^T \mathbf{P} + \mathbf{P} \mathbf{A}_M = -\mathbf{Q}, \quad (1.5)$$

where $\mathbf{Q}$ – arbitrary symmetric positive definite (in particular, diagonal) matrix.

Example of calculation of a system with direct adaptation and parametric adjustment for a first-order plant. Consider a plant in a form of PT₁-element

$$\begin{cases} \dot{x} = a_0 x + b_0 u \\ y_0 = x \end{cases}, \quad (1.6)$$

For this example, take a stable and unstable plants with the following parameters: $a_{0s} = -8, b_{0s} = 5, a_{0u} = 0.8, b_{0u} = 1$. We set desired plant transient function using reference model. As a reference model we also take a PT₁-element with following parameters: $a_m = -15, b_m = 15$. The transient responses of stable and unstable plants, as well as the reference model, are shown in Fig. 1.1.

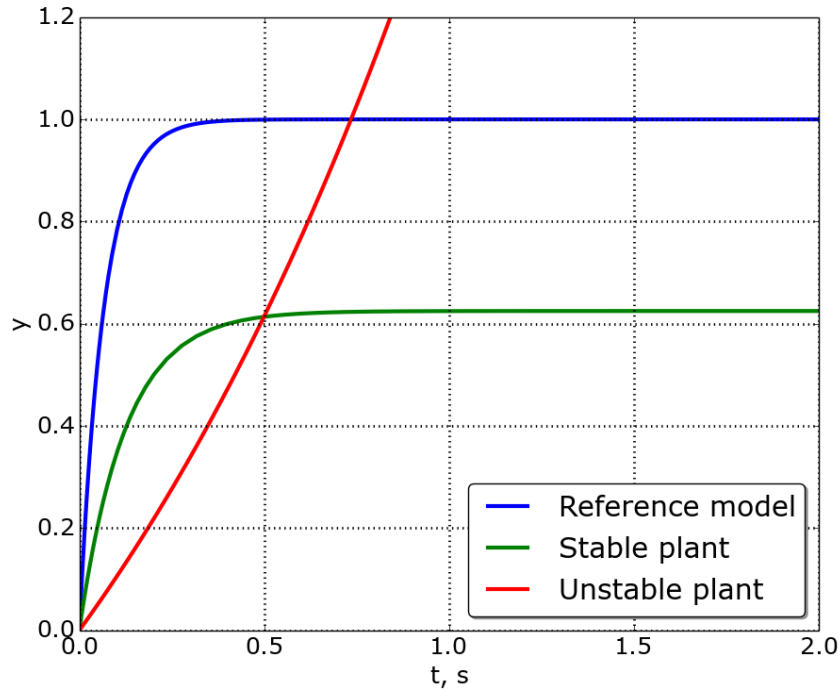


Fig. 1.1 Transient responses of stable and unstable plants and reference model

Let us now calculate adaptive control system with parametric adjustment. We take positive coefficient p equal to 10 and positive coefficients γ_A, γ_B take from interval from 1 to 100. Block diagram of the system with a direct adaptive control and a reference model is shown in Fig. 1.2.

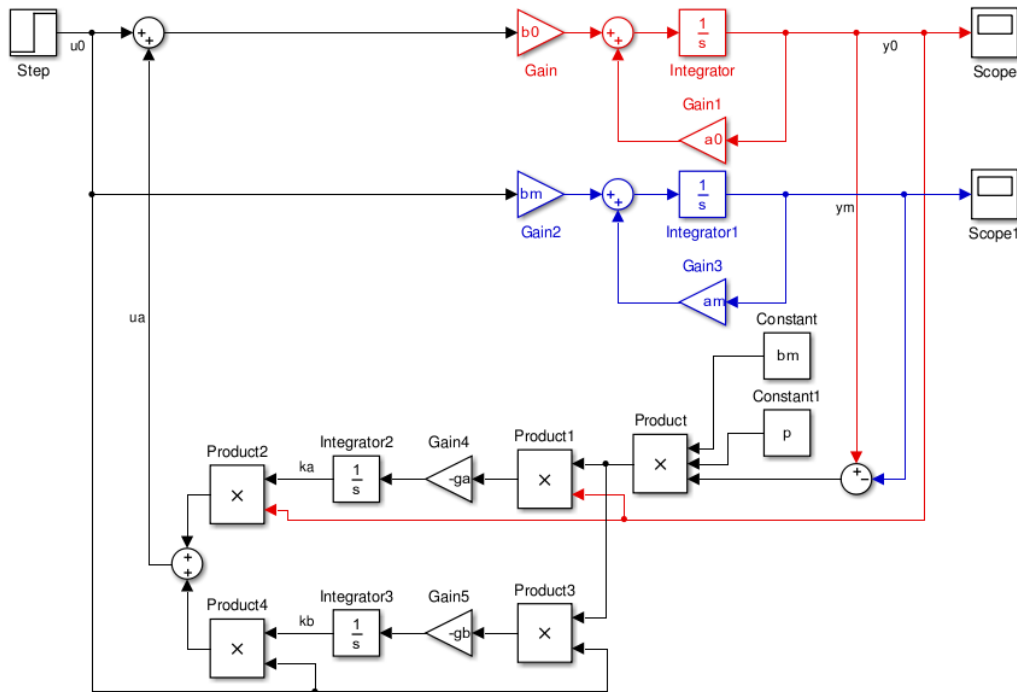


Fig. 1.2. Block diagram of the system with a direct adaptive control and a reference model

The transient responses of resulting system with stable plant with different values of coefficients γ_A, γ_B in comparison with reference model transient function are shown in Fig. 1.3.

The transient responses of resulting system with unstable plant with different values of coefficients γ_A, γ_B in comparison with reference model transient response are shown in Fig. 1.4.

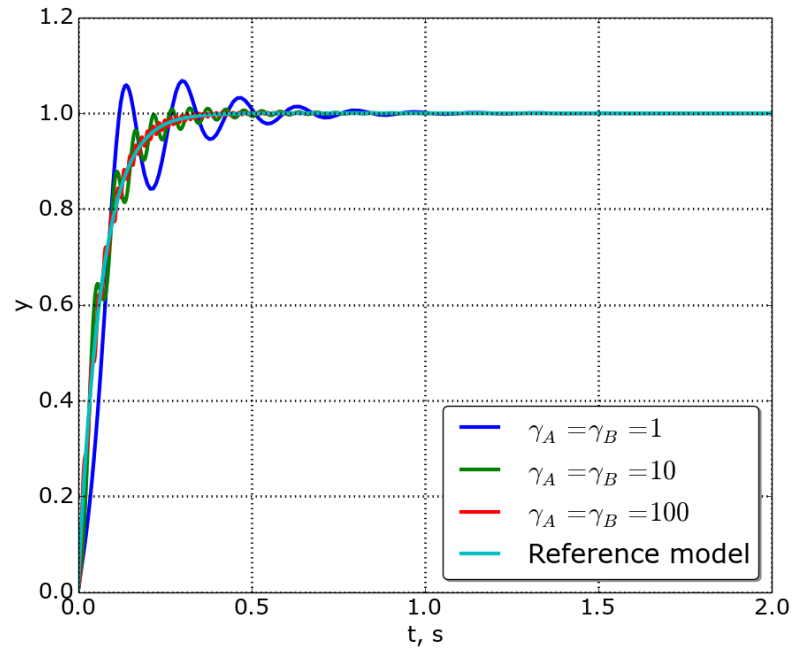


Fig. 1.3. The transient responses of resulting system with stable plant with different values of coefficients γ_A, γ_B in comparison with reference model transient response

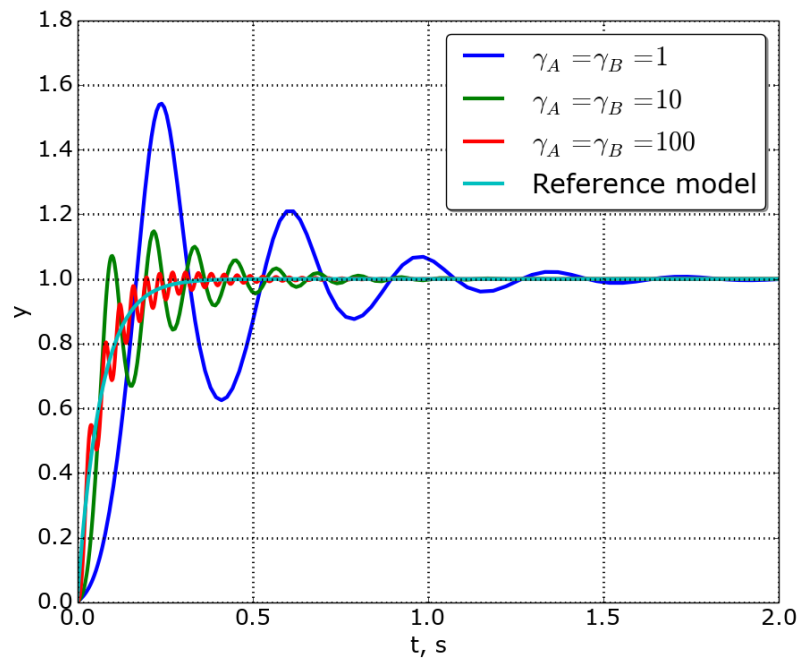


Fig. 1.4. The transient responses of resulting system with unstable plant with different values of coefficients γ_A, γ_B in comparison with reference model transient response

Laboratory work program.

1. Investigate system (1.1) with given parameters a_0, b_0 for stable plant ($a_0 < 0$).
2. Investigate reference model (1.3) and compare its dynamics with plant (1.1) dynamics.
3. Investigate adaptive control system (1.2) with adjustment algorithms (1.4) and given initial values of γ_A, γ_B, p .
4. Investigate adaptive system by varying its coefficients γ_A, γ_B ($p = \text{const}$).
5. Find steady values of $k_A \rightarrow k_A^{\text{steady}}$ and $k_B \rightarrow k_B^{\text{steady}}$ and compare them with values calculated using following formulas $k_A^* = b_0^{-1}(a_M - a_0)$, $k_B^* = b_0^{-1}(b_M - b_0)$.
6. Investigate system (1.1) with given parameters a_0, b_0 for unstable plant ($a_0 > 0$).
7. Repeat steps 3-5 for unstable plant.

Variant #1

Stable plant:

$$a_0 = -0.5, b_0 = 1, a_M = -5, b_M = 5, p = 2, \gamma_A = \gamma_B = 1$$

Unstable plant:

$$a_0 = 0.5, b_0 = 1, a_M = -5, b_M = 5, p = 2, \gamma_A = \gamma_B = 1$$

Variant #2

Stable plant:

$$a_0 = -3, b_0 = 1, a_M = -15, b_M = 10, p = 0.5, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 3, b_0 = 1, a_M = -15, b_M = 10, p = 0.5, \gamma_A = \gamma_B = 0.1$$

Variant #3

Stable plant:

$$a_0 = -2, b_0 = 1, a_M = -15, b_M = 7, p = 0.1, \gamma_A = \gamma_B = 0.01$$

Unstable plant:

$$a_0 = 2, b_0 = 1, a_M = -15, b_M = 7, p = 0.1, \gamma_A = \gamma_B = 0.01$$

Variant #4

Stable plant:

$$a_0 = -0.1, b_0 = 1, a_M = -1, b_M = 1, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 0.1, b_0 = 1, a_M = -1, b_M = 1, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Variant #5

Stable plant:

$$a_0 = -0.8, b_0 = 0.8, a_M = -8, b_M = 8, p = 1, \gamma_A = \gamma_B = 1$$

Unstable plant:

$$a_0 = 0.8, b_0 = 0.8, a_M = -8, b_M = 8, p = 1, \gamma_A = \gamma_B = 1$$

Variant #6

Stable plant:

$$a_0 = -0.15, b_0 = 1, a_M = -15, b_M = 5, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 0.15, b_0 = 1, a_M = -15, b_M = 5, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Variant #7

Stable plant:

$$a_0 = -0.1, b_0 = 1, a_M = -1, b_M = 1, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 0.5, b_0 = 1, a_M = -5, b_M = 5, p = 2, \gamma_A = \gamma_B = 1$$

Variant #8

Stable plant:

$$a_0 = -0.3, b_0 = 1, a_M = -1, b_M = 10, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 0.35, b_0 = 1, a_M = -12, b_M = 1, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Variant #9

Stable plant:

$$a_0 = -0.5, b_0 = 1, a_M = -5, b_M = 5, p = 2, \gamma_A = \gamma_B = 1$$

Unstable plant:

$$a_0 = 0.8, b_0 = 0.8, a_M = -8, b_M = 8, p = 1, \gamma_A = \gamma_B = 1$$

Variant #10

Stable plant:

$$a_0 = -3, b_0 = 1, a_M = -15, b_M = 10, p = 0.5, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 0.1, b_0 = 1, a_M = -1, b_M = 10, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Variant #11

Stable plant:

$$a_0 = -3.5, b_0 = 1, a_M = -17, b_M = 8, p = 2, \gamma_A = \gamma_B = 1$$

Unstable plant:

$$a_0 = 3.5, b_0 = 1, a_M = -17, b_M = 8, p = 2, \gamma_A = \gamma_B = 1$$

Variant #12

Stable plant:

$$a_0 = -1.5, b_0 = 1, a_M = -15, b_M = 15, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Unstable plant:

$$a_0 = 1.5, b_0 = 1, a_M = -15, b_M = 15, p = 0.1, \gamma_A = \gamma_B = 0.1$$

Report contents.

- Goal of the work.
- Theoretical background.
- Adaptive control system calculation for stable plant.

- Conclusions about the effectiveness of adaptive system with parametric adjustment for a stable plant control.
- Adaptive control system calculation for unstable plant.
- Conclusions about the effectiveness of adaptive system with parametric adjustment for an unstable plant control.

2. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS WITH PARAMETRIC ADAPTATION FOR SECOND-ORDER PLANTS

Goal of the work: mastering the skills of adaptive control systems investigation, studying the adaptive control effectiveness in presence of plant parameters changing, and researching the adaptive control systems ability to stabilize the plant.

Theoretical background. Equations of direct adaptation system with parametric adjustment are presented in Laboratory work #1.

Example of calculation of a system with direct adaptation and parametric adjustment for a second-order plant. Consider a plant in the following form

$$\dot{\mathbf{x}}(t) = \mathbf{A}_0 \mathbf{x}(t) + \mathbf{b}_0 u(t), \quad u(t) = u_A(t) + u^0(t), \quad (2.1)$$

where $\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ a_1 & a_2 \end{bmatrix}$, $\mathbf{b}_0 = \begin{bmatrix} 0 \\ b \end{bmatrix}$ – unknown constant matrices; $\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$ – plant state vector; $u(t)$ – scalar real-valued function; $u^0(t)$ – reference input, and $u_a(t)$ – adaptive control action.

Reference model has following form:

$$\dot{\mathbf{x}}_M(t) = \mathbf{A}_M \mathbf{x}_M(t) + \mathbf{b}_M u^0(t), \quad (2.2)$$

where $\mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ a_{M1} & a_{M2} \end{bmatrix}$, $\mathbf{b}_M = \begin{bmatrix} 0 \\ b_M \end{bmatrix}$, $\mathbf{x}_M(t) = \begin{bmatrix} x_{M1}(t) \\ x_{M2}(t) \end{bmatrix}$ – reference model state vector.

For this example, take the following plant and reference model parameters:

$a_1 = -0.8, a_2 = -3, b = 1, a_{m1} = -10, a_{m2} = -16, b_m = 16$. Plant and reference model transient responses are shown in Fig. 2.1.

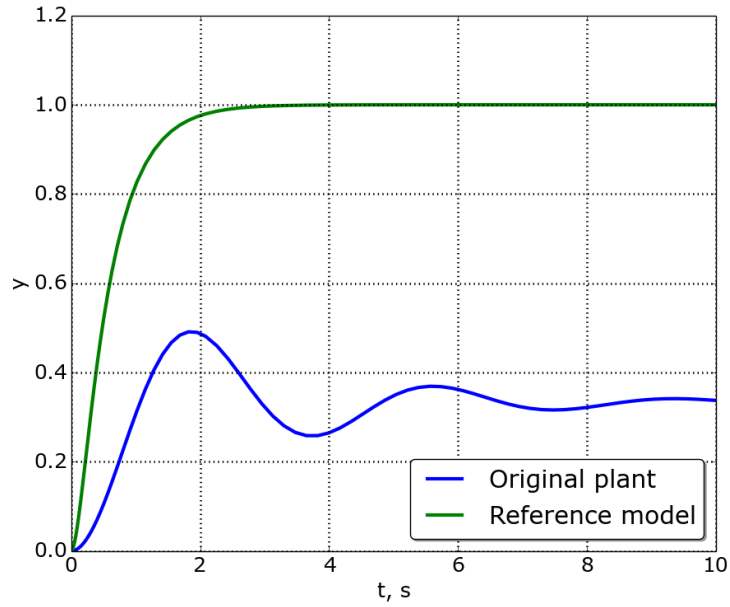


Fig. 2.1. Plant and reference model transient responses

Let us now calculate adaptive control system with parametric adjustment. We take constant positive definite matrix $\mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ and positive coefficients $\gamma_{a1}, \gamma_{a2}, \gamma_b$ take from interval from 1 to 100. Block diagram of the system with a direct adaptive control and a reference model is shown in Fig. 2.2.

The transient responses of resulting system with different values of coefficients $\gamma_{a1}, \gamma_{a2}, \gamma_b$ in comparison with reference model transient response are shown in Fig. 2.3.

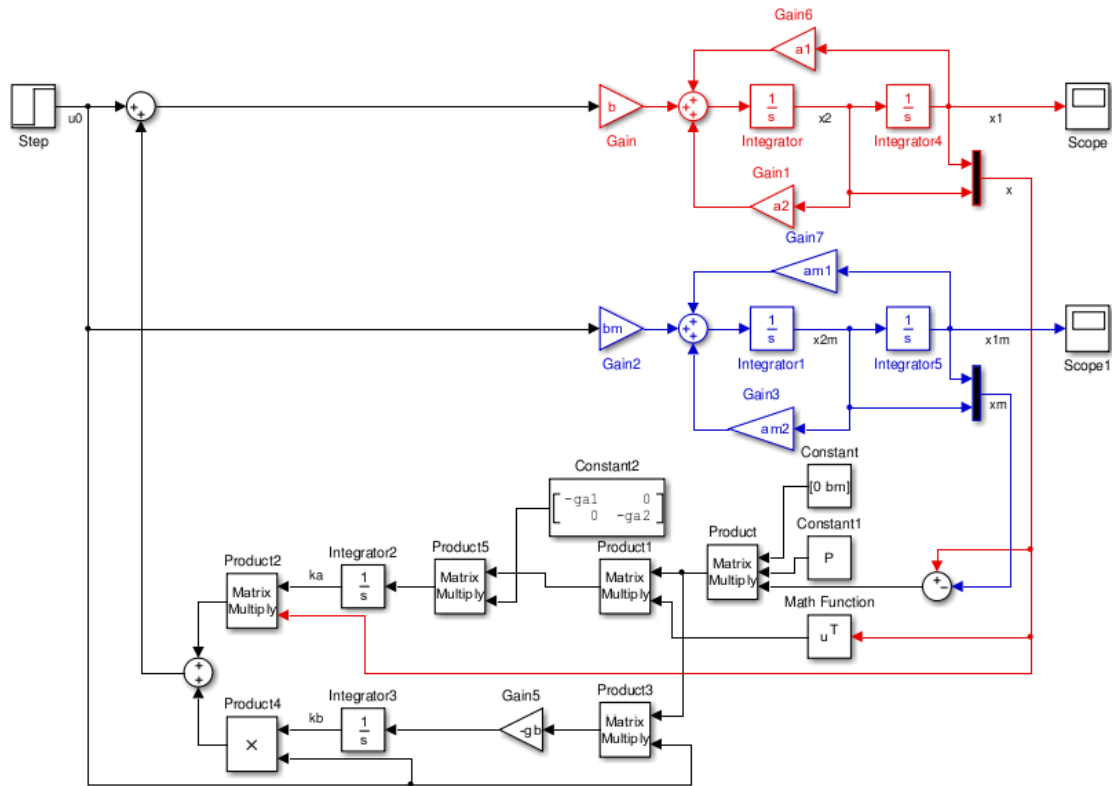


Fig. 2.2. Block diagram of the system with a direct adaptive control and a reference model is shown

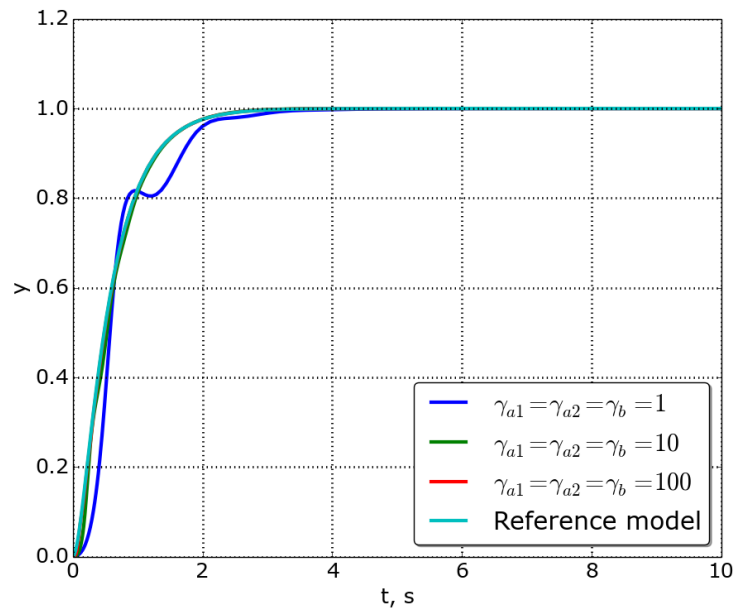


Fig. 2.3. The transient responses of resulting system with different values of coefficients $\gamma_{a1}, \gamma_{a2}, \gamma_b$ in comparison with reference model transient response

Laboratory work program.

1. Investigate system (2.1) with given matrices $\mathbf{A}_0, \mathbf{b}_0$ for stable plant.
2. Investigate reference model (2.2) and compare its dynamics with plant (2.1) dynamics.
3. Investigate adaptive control system (1.2) with adjustment algorithms (1.4) and given initial values of $\mathbf{\Gamma}_A, \gamma_B, \mathbf{P}$.
4. Investigate adaptive system by varying parameters of $\mathbf{\Gamma}_A, \gamma_B$ ($\mathbf{P} = \text{const}$).
5. Find steady values of $\mathbf{k}_A \rightarrow \mathbf{k}_A^{\text{steady}}$ and $k_B \rightarrow k_B^{\text{steady}}$ and compare them with values calculated using following formulas $\mathbf{k}_A^* = \mathbf{b}_0^+(\mathbf{A}_M - \mathbf{A}_0)$, $k_B^* = \mathbf{b}_0^+(\mathbf{b}_M - \mathbf{b}_0)$, where $\mathbf{b}_0^+ = (\mathbf{b}_0^T \mathbf{b}_0)^{-1} \mathbf{b}_0^T$ – Moore-Penrose pseudoinverse matrix.
6. Investigate system (1.1) with given matrices $\mathbf{A}_0, \mathbf{b}_0$ for unstable plant.
7. Repeat steps 3-5 for unstable plant.

Variant #1

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -2 & -4 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -10 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 2 & 4 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -10 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Variant #2

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -10 & -3 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -5 & -7 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \gamma_B = 2$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 10 & 3 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -5 & -7 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \gamma_B = 2$$

Variant #3

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -2 & -0.5 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -3 & -5 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 2 & 0.5 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -3 & -5 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Variant #4

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -0.3 & -0.8 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -15 & -25 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 40 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 20 & 10 \\ 10 & 10 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}, \gamma_B = 10$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 0.3 & 0.8 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -15 & -25 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 40 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 20 & 10 \\ 10 & 10 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}, \gamma_B = 10$$

Variant #5

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -3 & -1 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1,$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 3 & 1 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1,$$

Variant #6

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -5 & -2 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -32 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}, \gamma_B = 0.1$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 5 & 2 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -32 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}, \gamma_B = 0.1,$$

Variant #7

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -3 & -1 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1,$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 2 & 0.5 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Variant #8

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -10 & -3 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -5 & -7 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \gamma_B = 2$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 2 & 0.5 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -5 & -7 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Variant #9

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -0.3 & -0.8 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -15 & -25 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 40 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 20 & 10 \\ 10 & 10 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}, \gamma_B = 10$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 2 & 4 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -10 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Variant #10

Stable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ -2 & -4 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -10 & -16 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \gamma_B = 1$$

Unstable plant

$$\mathbf{A}_0 = \begin{bmatrix} 0 & 1 \\ 5 & 2 \end{bmatrix}, \mathbf{b}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{A}_M = \begin{bmatrix} 0 & 1 \\ -8 & -32 \end{bmatrix}, \mathbf{b}_M = \begin{bmatrix} 0 \\ 16 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \mathbf{\Gamma}_A = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}, \gamma_B = 0.1$$

Report contents.

- Goal of the work.
- Theoretical background.
- Adaptive control system calculation for stable plant.
- Conclusions about the effectiveness of adaptive system with parametric adjustment for a stable plant control.
- Adaptive control system calculation for unstable plant.
- Conclusions about the effectiveness of adaptive system with parametric adjustment for an unstable plant control.

3. SYNTHESIS AND INVESTIGATION OF SYSTEMS WITH FULL STATE FEEDBACK CONTROL AND STATE OBSERVER.

Goal of the work: mastering the skills of systems with full state feedback control and state observer investigation, studying the effectiveness of full state feedback control with state observer in presence of plant parameters changing.

Theoretical background. Full state feedback control. Let us consider linear time-invariant plant of n order with single input (1.1). If plant (1.1) is controllable, i.e. the rank of its controllability matrix ($\mathbf{R}=[\mathbf{B} \ \mathbf{AB} \ \mathbf{A}^2\mathbf{B} \ \dots \ \mathbf{A}^{n-1}\mathbf{B}]$) is equal to n , then it is possible to use full state feedback control in the following form:

$$u_l = \mathbf{k}^T \mathbf{x}, \quad (3.1)$$

where real vector $\mathbf{k} = (k_1 \ k_2 \ \dots \ k_n)^T$ contains feedback coefficients and is calculated using desired closed-loop system poles locations with only one restriction, that complex poles must be chosen with their conjugate values because all elements of $\mathbf{A}$, $\mathbf{b}$ matrices are real numbers. The characteristic polynomial of the system with full state feedback control (3.1) has the following form:

$$\varphi(\lambda) = \det(\mathbf{A} + \mathbf{bk}^T - \lambda\mathbf{I}), \quad (3.2)$$

where $\mathbf{I}$ is identity matrix. Desired closed-loop system characteristic polynomial is usually chosen in the form of standard Newton or Butterworth polynomials, which forms are presented in Table 3.1.

Table. 3.1

Order	Newton polynomial	Butterworth polynomial
1	$\lambda + \omega_0$	$\lambda + \omega_0$
2	$\lambda^2 + 2\omega_0\lambda + \omega_0^2$	$\lambda^2 + 1.4\omega_0\lambda + \omega_0^2$
3	$\lambda^3 + 3\omega_0\lambda^2 + 3\omega_0^2\lambda + \omega_0^3$	$\lambda^3 + 2\omega_0\lambda^2 + 2\omega_0^2\lambda + \omega_0^3$
4	$\lambda^4 + 4\omega_0\lambda^3 + 6\omega_0^2\lambda^2 + 4\omega_0^3\lambda + \omega_0^4$	$\lambda^4 + 2.6\omega_0\lambda^3 + 3.4\omega_0^2\lambda^2 + 2.6\omega_0^3\lambda + \omega_0^4$

All standard polynomials have ω_0 parameter. This parameter determines the distribution radius of system poles, so its value is always positive. All the roots of Newton polynomial are located at one point on the negative part of the real axis of the roots complex plane, and the roots of the Butterworth polynomial are located on the semicircle in the left half-plane of the roots complex plane.

One can calculate feedback coefficients of vector $\mathbf{k}$ by equating of the coefficients for the same λ powers in the closed-loop system characteristic polynomial (3.2) and the desired polynomial. For calculation simplicity one can use Matlab function “acker”, which has the following form: $\mathbf{K} = \mathbf{acker}(\mathbf{A}, \mathbf{B}, \mathbf{P})$, where $\mathbf{K}$ is calculated feedback coefficients, $\mathbf{A}$ and $\mathbf{B}$ are open-loop system matrices and $\mathbf{P}$ is a desired poles vector.

When full state feedback control is used, an additional normalizing coefficient $k_n = 1/(-\mathbf{c}/(\mathbf{A} - \mathbf{b}\mathbf{k})\mathbf{b})$ is also required, where $\mathbf{c}$ is a system (1.1) output matrix from its output equation $\mathbf{y} = \mathbf{c}\mathbf{x}$, and $\mathbf{y}$ is a system output.

Full-Order State Observer. To implement full state feedback control (as well as adaptive control with parametric adaptation) a full system state vector is required, but it is not always feasible to measure all system state variables. To obtain full state vector one can use asymptotic state variables estimates by using of full-order state observer. If plant (1.1) is observable, i.e. the rank of its observability matrix ($\mathbf{O} = [\mathbf{C} \ \mathbf{C}\mathbf{A} \ \mathbf{C}\mathbf{A}^2 \ \dots \ \mathbf{C}\mathbf{A}^{n-1}]^T$) is equal to n , then it is possible to use full-order state observer in the following form:

$$\dot{\hat{\mathbf{x}}} = \mathbf{A}\hat{\mathbf{x}} + \mathbf{b}u + \mathbf{l}(\mathbf{y} - \mathbf{c}\hat{\mathbf{x}}), \quad (3.3)$$

where $\hat{\mathbf{x}}$ is an state vector estimation, $\mathbf{l} = (l_1 \ l_2 \ ... \ l_n)^T$ is a vector providing required transient process quality for state vector estimation error. State observer characteristic polynomial has the following form:

$$\varphi(\lambda) = \det(\mathbf{A} + \mathbf{l}\mathbf{c}^T - \lambda\mathbf{I}). \quad (3.4)$$

Just as for full state feedback control, desired polynomial for state observer is chosen and coefficients of $\mathbf{l}$ vector are found by equating of the coefficients for the same λ powers in the state observer characteristic polynomial (3.4) and the desired polynomial.

It should be noted that, based on the independence of the characteristic polynomials of a closed-loop system with full state feedback control and the state observer, we can assign the characteristic polynomials roots for full state feedback control and the state observer independently of each other.

Example of calculation of a system with full state feedback control and full-order state observer. Let us consider second order plant in the following form:

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{b}\mathbf{u} \\ \mathbf{y} = \mathbf{c}\mathbf{x} \end{cases}, \quad (3.5)$$

where $\mathbf{A} = \begin{bmatrix} 0 & a_1 \\ -a_2 & -a_2 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 0 \\ a_2 \end{bmatrix}$ and $\mathbf{c} = [1 \ 0]$.

For this example we take following values for plant parameters: $a_1 = 3$, $a_2 = 2$. Plant step response is shown in Fig. 3.1.

Let us calculate full state feedback control coefficients, for this we use Newton polynomial with $\omega_0 = 4.5$. In this case system poles have following form: $\mathbf{p} = [-4.5 \ -4.5]$. Using Matlab “acker” function we calculate full state feedback control coefficients: $\mathbf{k} = \text{acker}(\mathbf{A}, \mathbf{b}, \mathbf{p})$. Full state feedback coefficients have the following values $\mathbf{k} = [2.375 \ 3.5]$ and normalizing coefficient $k_n = 3.375$.

To calculate state observer we also choose Newton polynomial. State observer dynamics should be about 3-5 times faster than full state feedback control system dynamics, so we take $\omega_0 = 13.5$. Using Matlab “acker” function we calculate state observer coefficients $\mathbf{l} = \text{acker}(\mathbf{A}', \mathbf{c}', \mathbf{p})$. State observer coefficients have the fol-

lowing values $\mathbf{I} = \begin{bmatrix} 25 & 42.0833 \end{bmatrix}$. Block diagram of the system with full state feedback control and full-order state observer is shown in Fig. 3.2.

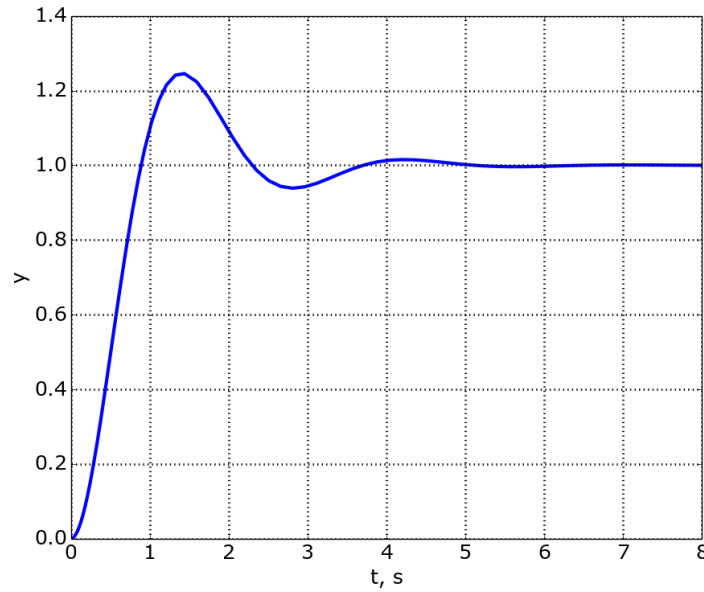


Fig. 3.1. Plant step response.

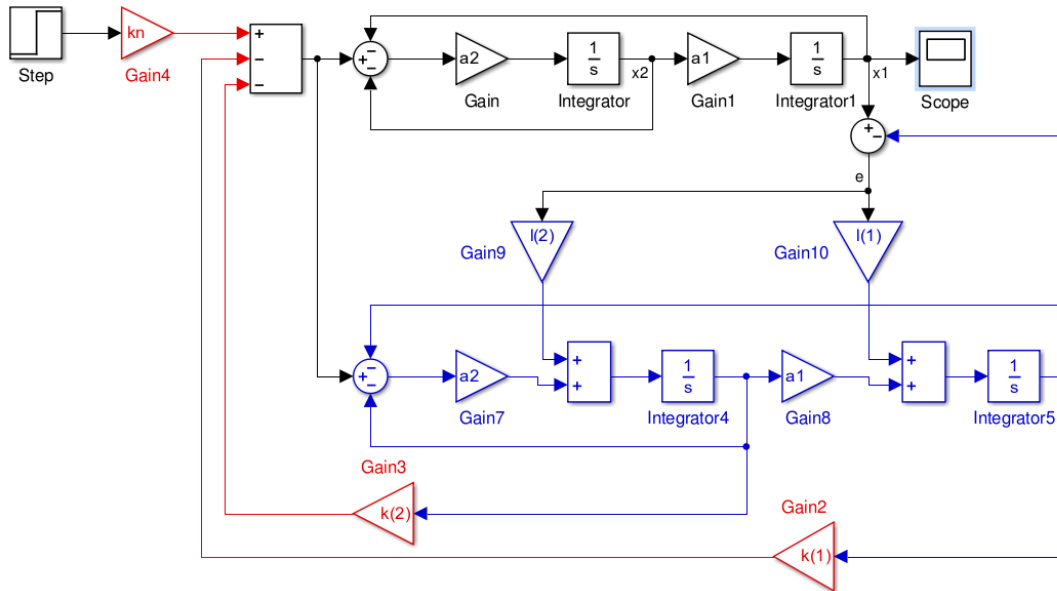


Fig. 3.2 Block diagram of the system with full state feedback control and full-order state observer

System step responses with nominal parameters, with a_1 increased in 3 times and with a_2 decreased in 3 times are presented in Fig. 3.3.

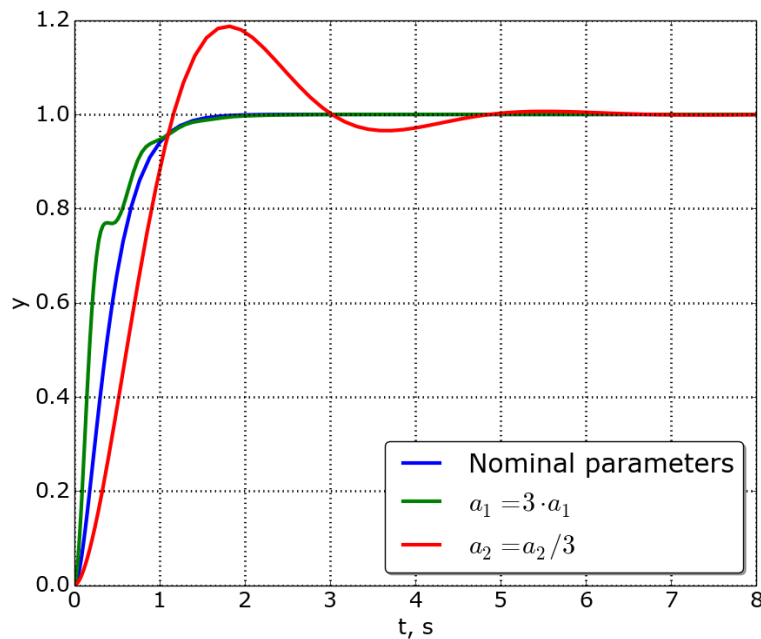


Fig. 3.3. Step responses of the system with full state feedback control and full-order state observer

Laboratory work program.

1. Investigate system (3.5) with given parameters a_1 and a_2 .
2. Calculate full state feedback control (3.1) and investigate fully measured system with nominal parameters and increased and decreased parameters a_1 and a_2 in 3 times.
3. Calculate full-order state observer (3.3) and investigate system with full state feedback control and state observer with nominal parameters and increased and decreased parameters a_1 and a_2 in 3 times.
4. Investigate system with full state feedback control and state observer with nominal parameters and deviations of state variables initial values from zero in range $[-0.5 \ 0.5]$.

Variant #1

$$\mathbf{A} = \begin{bmatrix} 0 & 5 \\ -1 & -1 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #2

$$\mathbf{A} = \begin{bmatrix} 0 & 2 \\ -3 & -3 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #3

$$\mathbf{A} = \begin{bmatrix} 0 & 2 \\ -1 & -1 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #4

$$\mathbf{A} = \begin{bmatrix} 0 & 2 \\ -4 & -4 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #5

$$\mathbf{A} = \begin{bmatrix} 0 & 8 \\ -4 & -4 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #6

$$\mathbf{A} = \begin{bmatrix} 0 & 6 \\ -4 & -4 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #7

$$\mathbf{A} = \begin{bmatrix} 0 & 8 \\ -1 & -1 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Variant #8

$$\mathbf{A} = \begin{bmatrix} 0 & 6 \\ -3 & -3 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Report contents.

- Goal of the work.
- Theoretical background.
- Full state feedback control system with state observer calculation.
- Investigation of full state feedback control system with state observer and varying of plant parameters and initial state.
- Conclusions about the effectiveness of full state feedback control with state observer.

4. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS FOR DOUBLE-MASS ELASTIC ELECTROMECHANICAL SYSTEM

Goal of the work: mastering the skills of systems with adaptive and full state feedback control, studying the effectiveness of adaptive and full state feedback control in presence of plant parameters changing.

Theoretical background. Linearized mathematical model of a double-mass elastic electromechanical system. Let us consider a linearized mathematical model of a double-mass elastic electromechanical system with permanent magnet DC motor without taking into account friction and backlash and control of second mass rotation velocity:

$$\left\{ \begin{array}{l} \dot{\omega}_2 = \frac{1}{I_2} \tau_e, \\ \dot{\tau}_e = p(\omega_1 - \omega_2), \\ \dot{\omega}_1 = \frac{k_t}{I_1} i - \frac{1}{I_1} \tau_e, \\ \dot{i} = \frac{1}{L} (u_\Sigma - k_e \omega_1 - Ri), \\ u_\Sigma = u_0 + u_l + u_a, \end{array} \right. \quad (4.1)$$

where ω_2 – working element (second mass) rotation velocity, I_2 – working element moment of inertia, τ_e – elasticity torque, arising in the elastic link due deformations, p – link stiffness, ω_1 – motor (first mass) shaft rotation velocity, k_e – motor back EMF constant determined by the flux density of the permanent magnets, k_t – motor torque constant that is dependent on the flux density of the permanent magnets, I_1 – motor moment of inertia, i – DC motor armature current, L – DC motor armature inductance, R – DC motor armature resistance, u_Σ – scalar control action, u_0 – reference input, u_l – full state feedback (linear) control action, u_a – adaptive control action.

Equations of direct adaptation system with parametric adjustment are presented in Laboratory work #1. Equations of full state feedback control system are presented in Laboratory work #3.

Example of calculation of an adaptive and full state feedback control system for double-mass elastic electromechanical system. For this example let us take the following plant parameters: $R = 2 \Omega$, $k_e = 1 \text{ V} \cdot \text{s/rad}$, $k_t = 1 \text{ N} \cdot \text{m/A}$, $I_1 = 0.05 \text{ kg} \cdot \text{m}^2$, $I_2 = 0.1 \text{ kg} \cdot \text{m}^2$, $L = 0.01 \text{ H}$, $p = 0.8 \text{ N} \cdot \text{m/rad}$. Original system step response is presented in Fig. 4.1.

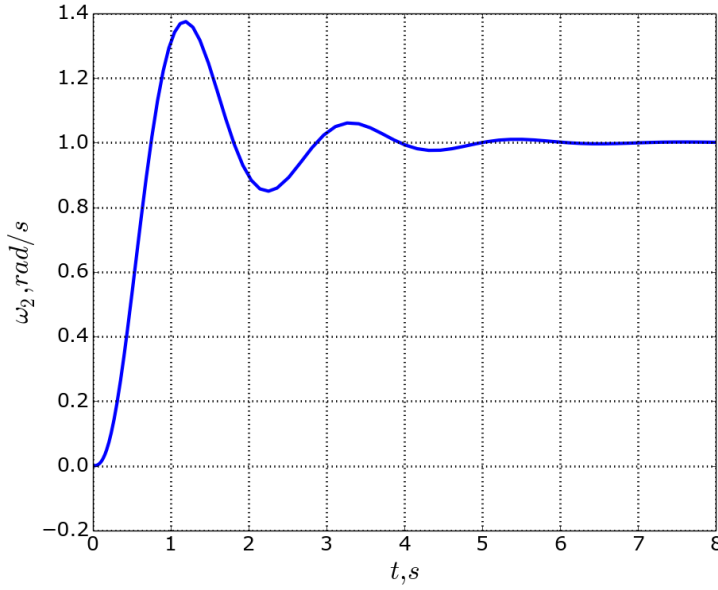


Fig. 4.1. Original system step response

Let us add a full state feedback control to the system. Original plant has the following representation in a state space:

$$\begin{bmatrix} \dot{\omega}_2 \\ \dot{\tau}_e \\ \dot{\omega}_1 \\ \dot{i} \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{I_2} & 0 & 0 \\ -p & 0 & p & 0 \\ 0 & -\frac{1}{I_1} & 0 & \frac{k_t}{I_1} \\ 0 & 0 & -\frac{k_e}{L} & -\frac{R}{L} \end{bmatrix} \begin{bmatrix} \omega_2 \\ \tau_e \\ \omega_1 \\ i \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{L} \end{bmatrix} u_\Sigma,$$

using **acker** function let us calculate full state feedback control vector $\mathbf{k}$. For this case choose Newton polynomial as a desired one with $\omega_0 = 7.5$. In this case a full state feedback control vector is $\mathbf{k} = [0.041 \ 0.6047 \ -0.8433 \ -1.7]$ and normalizing coefficient $k_n = 0.1978$. Fig. 4.2. shows original system and system with full state feedback control step responses.

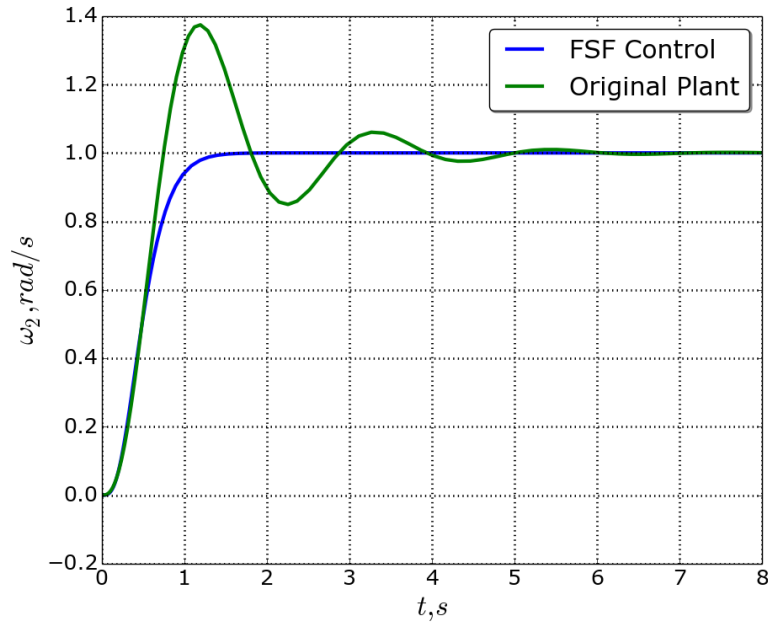


Fig. 4.2. Original system and system with full state feedback control step responses
As can be seen from step responses, the use of full state feedback control allows achieving desired system behavior.

Consider now system behavior with plant parameters changes. Fig. 4.3 shows system with full state feedback control step responses with a decrease in working element moment of inertia and link stiffness in 3 times.

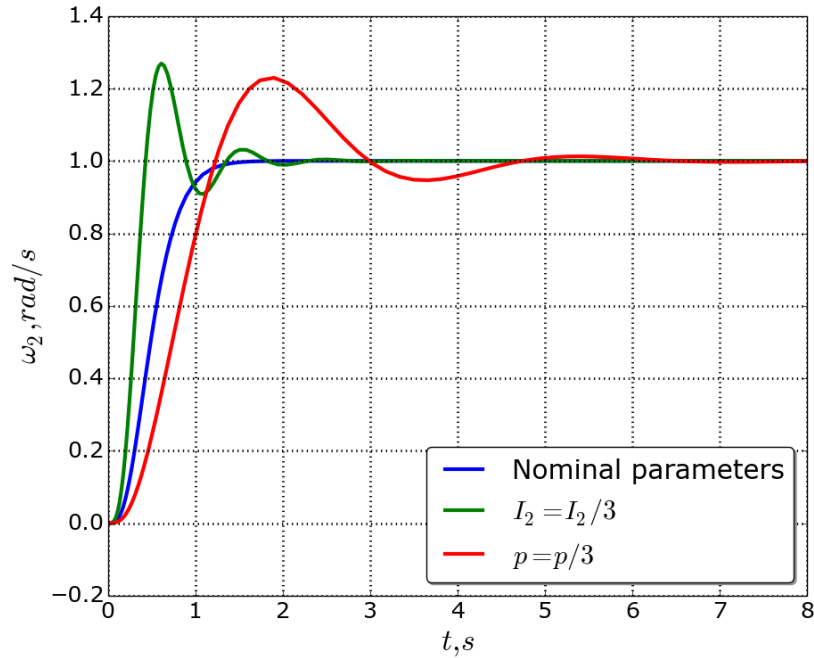


Fig. 4.3. System with plant parameters changes step responses

Let us now add a direct adaptation system with parametric adjustment and reference model. We take system with nominal parameters and full state feedback control as a reference model. In order to calculate $\mathbf{P}$ matrix we use the Lyapunov equation (1.5) and take as an arbitrary symmetric positive definite matrix $\mathbf{Q}$ the following matrices: $\mathbf{Q}_1 = \text{diag}[1,1,1,1]$, $\mathbf{Q}_2 = \text{diag}[100,10,10,100]$, $\mathbf{Q}_3 = \text{diag}[1000,1,1,1]$. To solve the Lyapunov equation, we can use Matlab **lyap** function that has the following form: $\mathbf{P} = \text{lyap}(\mathbf{A}_M', \mathbf{Q})$, where $\mathbf{P}$ – is desired $\mathbf{P}$ matrix, $\mathbf{A}_M'$ – is transpose reference model state matrix ($\mathbf{A}_M = \mathbf{A} - \mathbf{b}\mathbf{k}$). As a result of **lyap** function we get the corresponding square symmetric positive definite matrices $\mathbf{P}_1$, $\mathbf{P}_2$ and $\mathbf{P}_3$. Also we take $\mathbf{\Gamma}_A = \text{diag}([100,100,100,100])$ and $\gamma_B = 100$. Block diagram of the system with full state feedback and adaptive control with reference model is presented in Fig. 4.4.

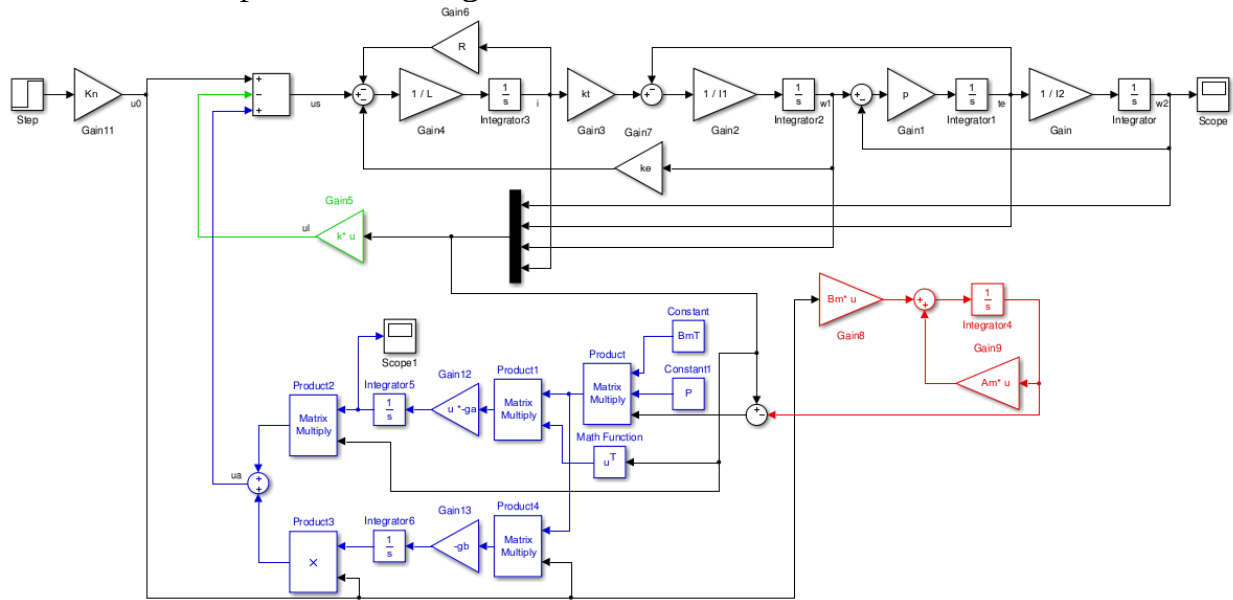


Fig. 4.4 Block diagram of the system with full state feedback and adaptive control with reference model

Step responses of the system with full state feedback and adaptive control with reference model and reduced in 3 times working element moment of inertia is shown in Fig. 4.5.

Step responses of the system with full state feedback and adaptive control with reference model and reduced in 3 times link stiffness is shown in Fig. 4.6.

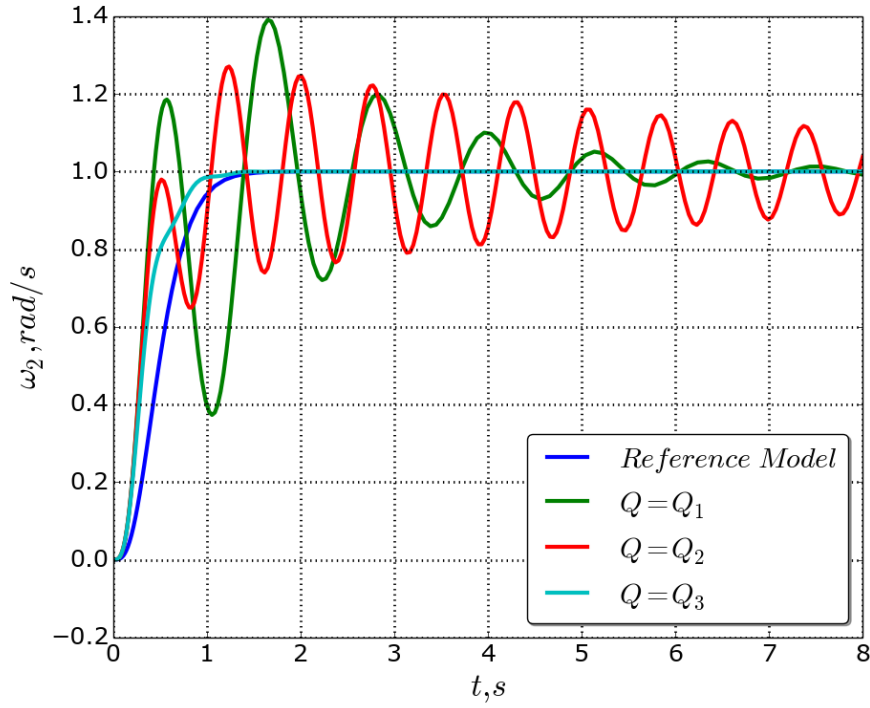


Fig. 4.5. Step responses of the system with full state feedback and adaptive control with reference model and reduced in 3 times working element moment of inertia

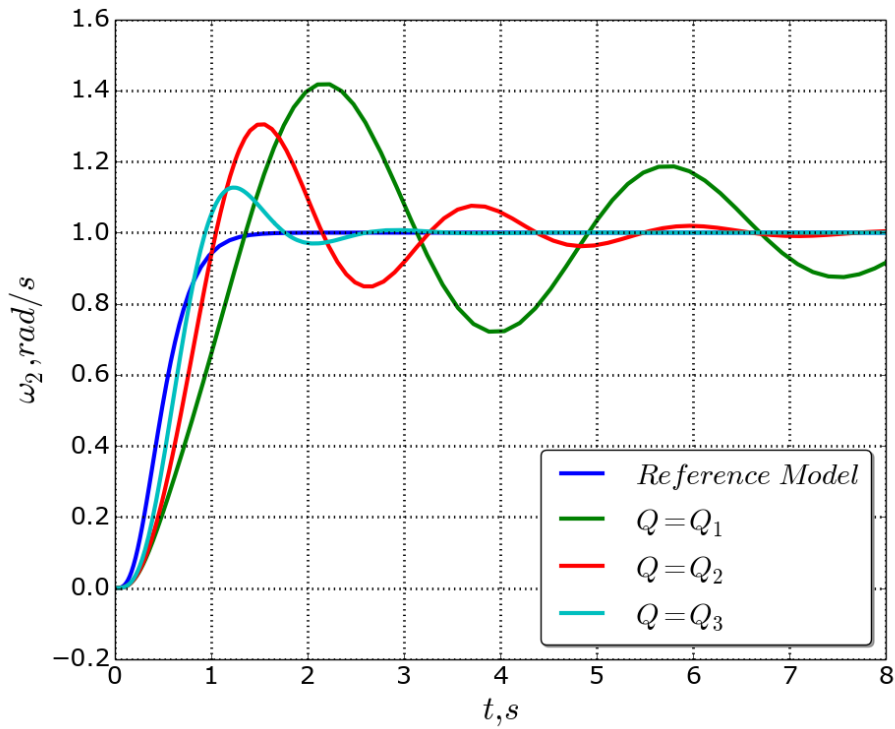


Fig. 4.6. Step responses of the system with full state feedback and adaptive control with reference model and reduced in 3 times link stiffness

As can be seen from system step responses, step response quality depends on the chosen matrix $\mathbf{Q}$. In our example, for 3 times parameters changes matrix $\mathbf{Q}_3$ shows best results.

Laboratory work program.

1. Investigate system (4.1) with given parameters p and I_2 .
2. Calculate full state feedback control (3.1) and investigate fully measured system with nominal parameters and increased and decreased parameters p and I_2 in 3 times.
3. Add adaptive control system (1.2) with adjustment algorithms (1.4) with different square symmetric positive definite matrices $\mathbf{Q}$ and investigate obtained system with nominal parameters and increased and decreased parameters p and I_2 in 3 times.
4. Compare steady state values of $\mathbf{k}_A$ and k_B in system with full state feedback and adaptive control and system with only adaptive control.

Variant #1

$$p = 0.5 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.1 \text{ kg} \cdot \text{m}^2$$

Variant #2

$$p = 0.8 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.2 \text{ kg} \cdot \text{m}^2$$

Variant #3

$$p = 0.6 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.15 \text{ kg} \cdot \text{m}^2$$

Variant #4

$$p = 0.4 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.08 \text{ kg} \cdot \text{m}^2$$

Variant #5

$$p = 0.7 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.12 \text{ kg} \cdot \text{m}^2$$

Variant #6

$$p = 0.8 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.08 \text{ kg} \cdot \text{m}^2$$

Variant #7

$$p = 1 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.3 \text{ kg} \cdot \text{m}^2$$

Variant #8

$$p = 0.3 \text{ N} \cdot \text{m} / \text{rad}, I_2 = 0.05 \text{ kg} \cdot \text{m}^2$$

Report contents.

- Goal of the work.
- Theoretical background.
- Full state feedback control calculation.
- Investigation of full state feedback control system with nominal and increased and decreased plant parameters.
- Adaptive control system calculation.
- Investigation of full state feedback and adaptive control system with nominal and increased and decreased plant parameters.
- Conclusions about the effectiveness of full state feedback control and full state feedback and adaptive control.

5. GENETIC ALGORITHMS

Goal of the work: studying genetic algorithms basics, mastering the skills of applying genetic algorithms in Matlab.

Theoretical background. Genetic algorithms. Genetic algorithms are stochastic methods of search and optimization. They are based on the principles of natural selection and evolution. Genetic algorithms work with populations of specially-coded potential solutions of the problem and apply to them the principles of survival of the strongest individuals, i.e. most suitable solutions. Coding takes place in the form of chromosomes, which are lines of genes. Each solution is tested by fit-

ness function. The task of genetic algorithms is to find a solution that reaches the extremum of the fitness function. Genetic algorithms are iterative algorithms, at each iteration of which a new population of individuals is created by selecting the most adapted ones and applying to them genetic operators. This process leads to the evolution of potential solutions and an increase in the average fitness of the population. In comparison with gradient search methods, such as the gradient descent, the Newton method and the Levenberg-Marquardt algorithm, one can highlight the following properties of genetic algorithms:

1. Genetic algorithms do not work particular problem parameters, but with their encoded forms;
2. Search starts not from a single point, but from a population;
3. Only fitness function is used, genetic algorithms do not need its derivatives or any other additional information;
4. Probabilistic, not deterministic, rules of choice are used.

Genetic algorithms are global search methods and because only the fitness function is used, they can be applied to tasks that cannot be solved with strict and formal analysis.

Genetic algorithm consists of the following stages:

1. Initialization;
2. Calculation individuals' fitness functions;
3. Stopping criteria checking;
4. Selection of individuals;
5. Application of genetic operators;
6. New population building and return to stage 2.

Let's consider in more detail the work of the genetic algorithm at each stage. To do this, we introduce the necessary concepts from genetics.

Population is a finite set of individuals.

Gene is an element of a chromosome that represents a particular property of an individual.

Fitness function is a mathematical function representing the measure of fitness of an individual in a population.

In order to use genetic algorithm it is necessary to specify a way to encode problem solutions in the form of a chromosome. The most common ways to encode of individual genes are binary or real numbers.

At the initialization stage, the initial population is created in the form of a random set of individuals, which are sequences of genes of fixed length (chromosomes).

At the stage of the individuals' fitness calculation, the fitness of each individual in the population is calculated using the fitness function.

At the stopping criteria checking stage, the criteria for completing the algorithm are checked. The most common criteria are:

- Achievement of the certain value of fitness function;
- Stagnation of the population best fitness function value at the same value for a certain number of iterations;
- Execution of a certain number of algorithm iterations.

At the selections stage based on calculated fitness functions values individuals are selected to participate in the next population creation process. Selections are made according to natural selection principles, individuals with the best values of fitness function have greater chance of taking part in new individuals creation. As a result of the selection stage, a parent pool is formed, from which the parent pairs are selected.

At the application of genetic operators stage genetic operators are applied to formed parent pairs. In genetic algorithms, two main genetic operators are used: crossover and mutation. Usually the mutation plays a less important role in comparison with crossover, so crossover occurs at almost every algorithm iteration, and the mutations are quite rare. Offspring pairs are obtained using crossover. Mutation can be applied both to the parents before crossover and to the offspring. Mutation is a random variation of the individuals chromosomes. Mutation is used in order to move the population from the local extremum and prevent premature convergence of the algorithm. At the new population building stage individuals from parent pool and offspring are selected to form a new population.

Average population fitness is increasing in genetic algorithms, and due to the large number of individuals and a presence of mutation, a large search area is provided. Due to these properties, genetic algorithms have found applications in many fields of science and industry, including control systems. In control systems, genetic algorithms are used in problems of automatic control, identification and analysis of stability. In some cases, genetic algorithms are used standalone, in other cases genetic algorithms are combined with analytical and intellectual control theory methods.

Example of using of genetic algorithms in Matlab for function minimum search. Let us consider as an example the search for a minimum of a nonlinear function of the form:

$$f(x, y) = \sin(y)e^{(1-\cos x)^2} + \cos(x)e^{(1-\sin y)^2} + (x - y)^2 \quad (5.1)$$

that has the following bounds for its parameters:

$$\begin{aligned} -10 &\leq x \leq 0 \\ -6.5 &\leq y \leq 0 \end{aligned} \quad (5.2)$$

Function (5.1) surface plot is shown in Fig. 5.1.

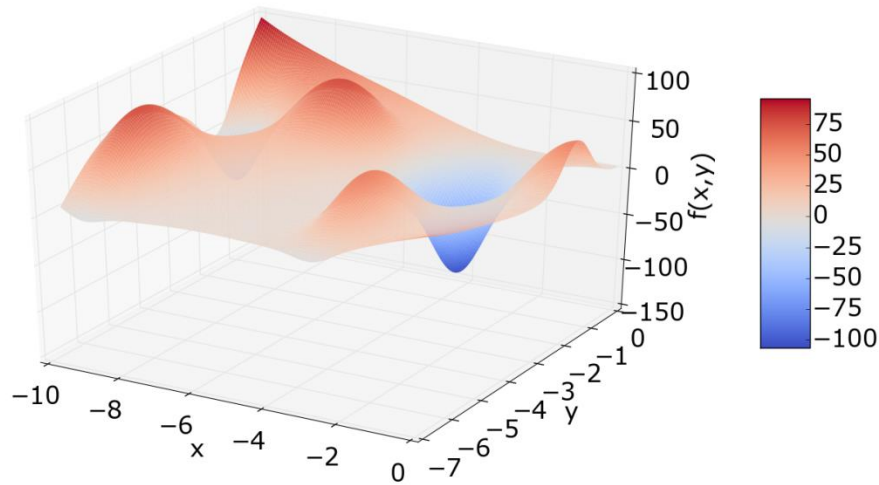


Fig. 5.1. $f(x,y)$ surface plot

Let us use Matlab in order to find function (5.1) minimum with a help of genetic algorithms. At first, one needs to prepare fitness function for genetic algorithm. To do that, we create a new .m file containing the following function that calculates the value of function (5.1):

```
function y = test_fitness(x)
y = sin(x(2)) * exp((1 - cos(x(1))) ^ 2) + cos(x(1)) *
exp((1 - sin(x(2))) ^ 2) + (x(1) - x(2)) ^ 2;
```

This function must return scalar value and take as an argument function parameters array.

Further, for simplicity, we will use a Matlab graphical interface for optimization algorithms. To launch it type in the Command Window area `optimtool('ga')` command and new window will be opened. Its appearance is shown in Fig. 5.2.

This window has 3 sections: 1. Problem Setup and Results, 2. Options and 3. Quick Reference. In “Fitness function” field it is necessary to specify our fitness function handler (in our example `@test_fitness`), in “Number of variables” field it is necessary to specify the number of function parameters (in our example it is 2). In the fields “Bounds” “Lower” and “Upper” we need to specify parameters bound constraints (in our example `[-10 -6.5]` and `[0 0]`). At first let us do not change default algorithm options, but only turn on display of best and average fitness function (checkbox “Best fitness” in the “Plot functions” options) and start the optimization process. The resulting optimization process is shown in Fig. 5.3. (It must be kept in mind that due to stochastic nature of genetic algorithms, the optimization result will differ from start to start, therefore it is recommended to run the optimization several times and select best result).

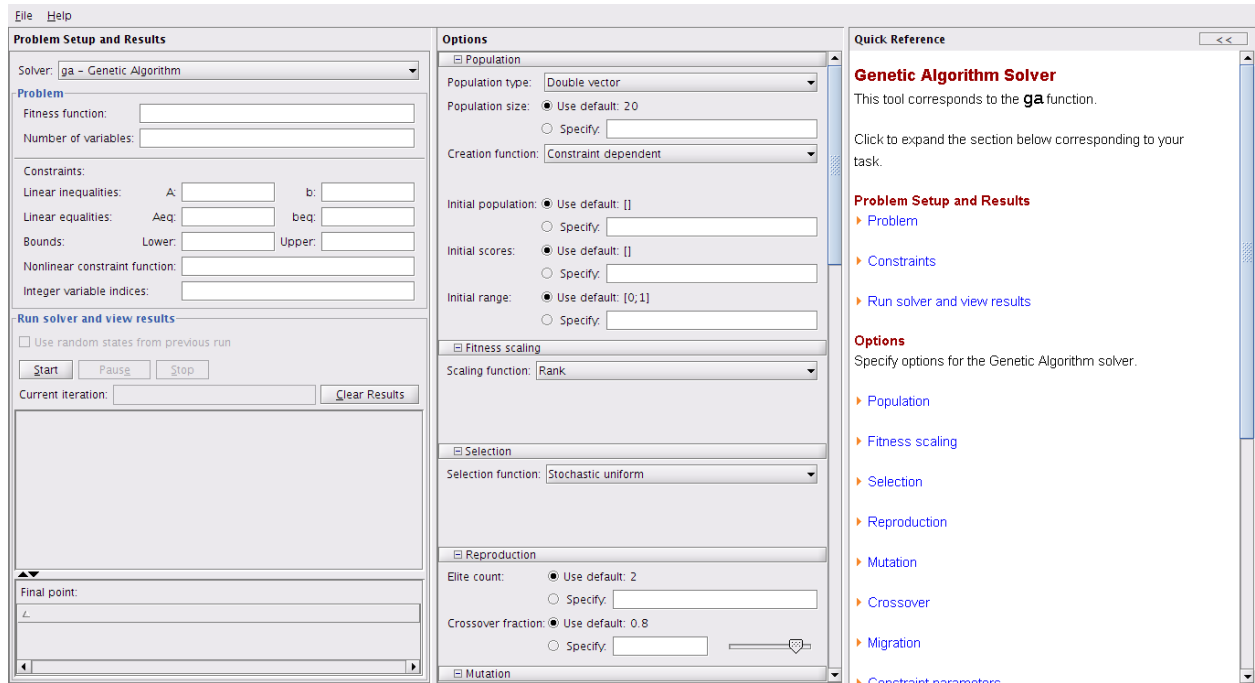


Fig. 5.2. Optimization window

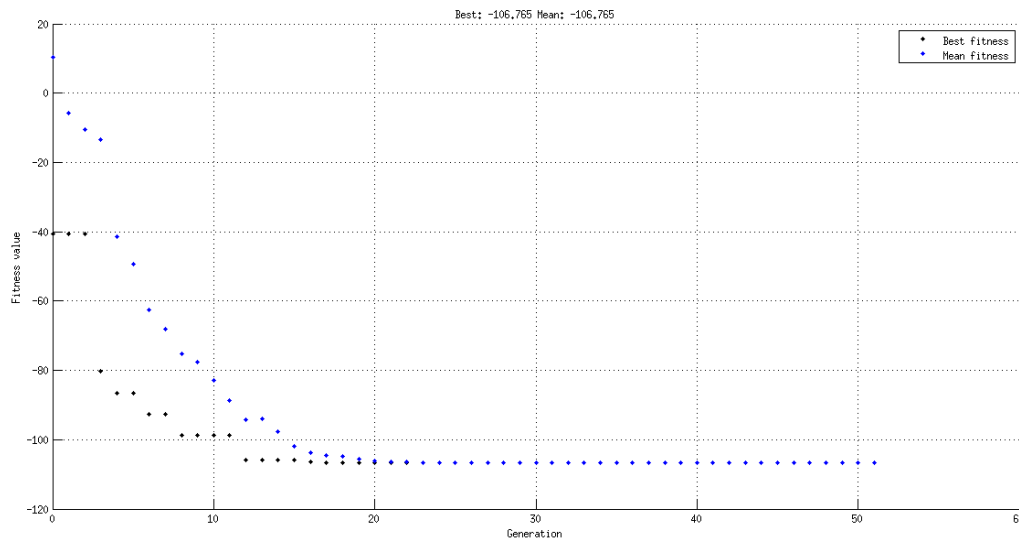


Fig. 5.3. Optimization process

As can be seen from optimization process, genetic algorithm successfully found function (5.1) global minimum. Let us find a function (5.1) minimum in a presence of nonlinear constraint of the following form:

$$x \cdot y - 13.8 = 0.$$

To specify nonlinear constraints, it is required to create additional function that gives nonlinear constraints in the form of the following equations and inequalities:

$$\begin{cases} ceq = 0 \\ c \leq 0 \end{cases}$$

In our example this function has the following form:

```
function [c, ceq] = test_constraint(x)
c = [];
ceq = [x(1) * x(2) - 13.8];
```

In this function we need to return 2 arrays with constraints. However, if there are no constraints of some form in the system, then an empty array should be returned. Genetic algorithm may not satisfy all the nonlinear constraints at every generation. If genetic algorithm converges to a solution, the nonlinear constraints will be satisfied at that solution. Function handler for nonlinear constraints calculation should be written in the “Nonlinear constraint function” field. For this example we will increase population size from 20 to 60 individuals and also we select the “Adaptive feasible” mutation and start optimization process again. The resulting optimization process is shown in Fig. 5.4.

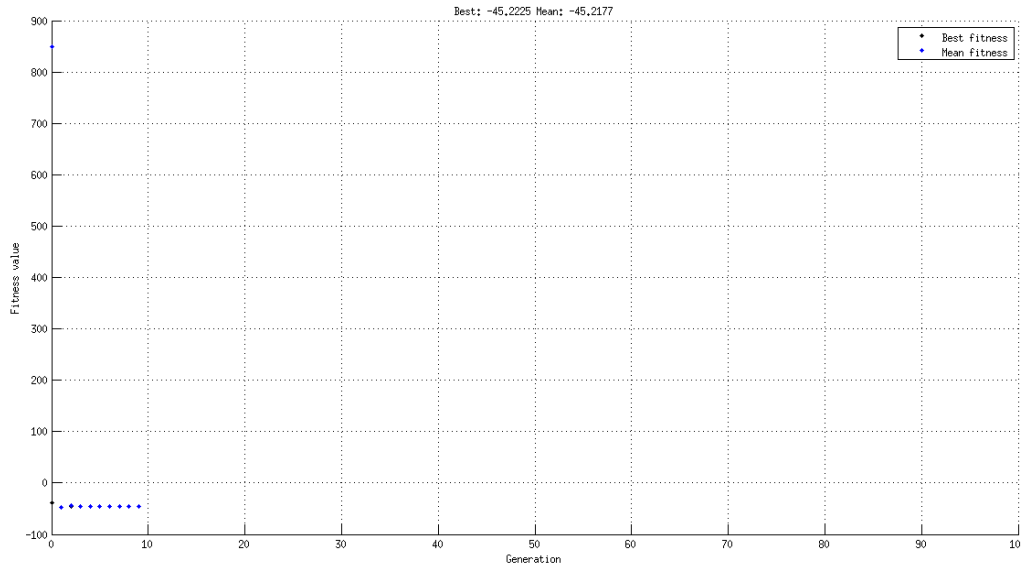


Fig. 5.4. Optimization process

As can be seen from optimization process, genetic algorithm successfully found function (5.1) global minimum in presence of nonlinear constraints.

Let us consider more difficult example:

$$f(x, y) = -\cos(x) \cdot \cos(y) \cdot e^{-((x-\pi)^2 + (y-\pi)^2)} \quad (5.3)$$

Function (5.3) surface plot is shown in Fig. 5.5.

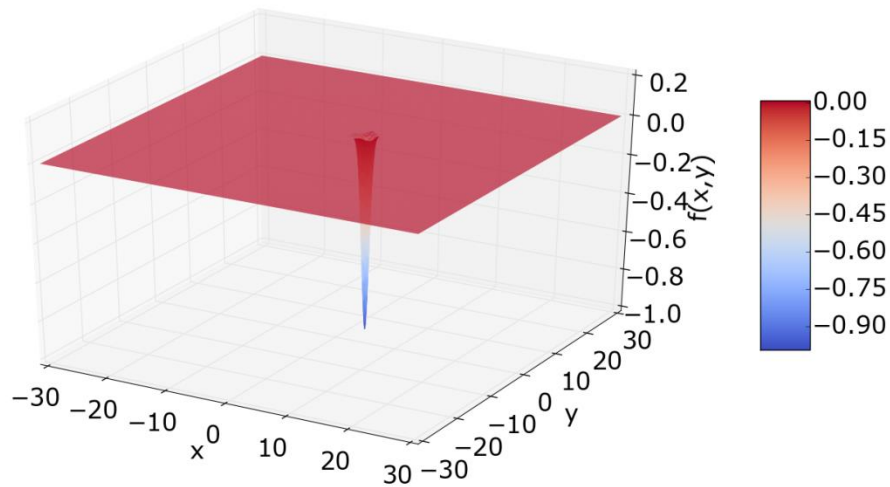


Fig. 5.5. Function (5.3) surface plot

Optimization process for function (5.3) is shown in Fig. 5.6.

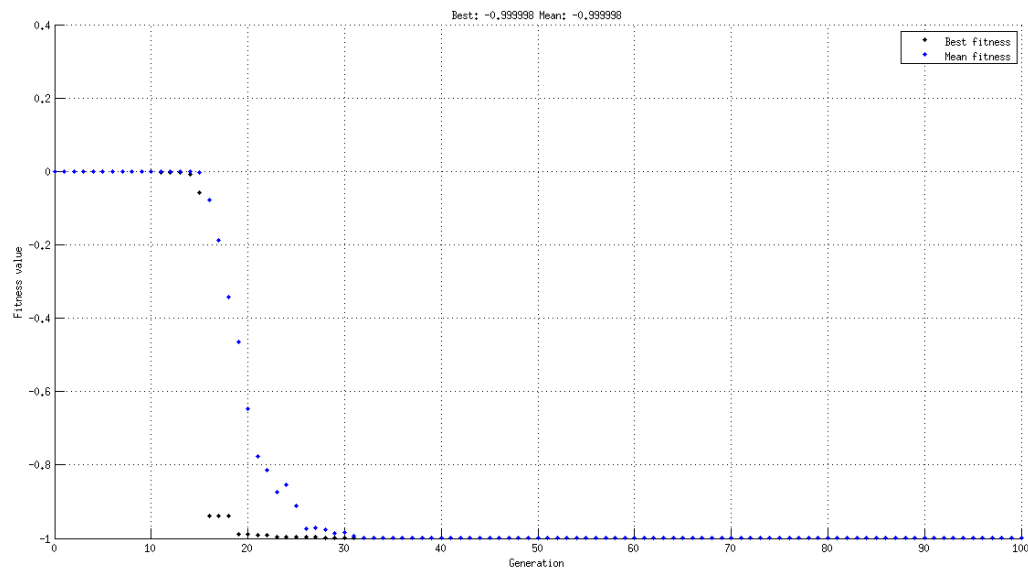


Fig. 5.6. Optimization process

As can be seen from optimization process, genetic algorithm successfully found function (5.3) global minimum.

Laboratory work program.

1. Using Matlab and genetic algorithms find minimum of scalar function:

$$f(x) = 8x - 16 - 12\sqrt[3]{(x+4)^2}, x \geq -4$$

2. Using Matlab and genetic algorithms find maximum of function $z(x, y) = \exp(-x^2 - y^2) + \sin(x + y)$. (To find the maximum, problem can be transformed into a minimum search for function $-z(x, y)$). Function $z(x, y)$ surface plot is shown in Fig. 5.7.

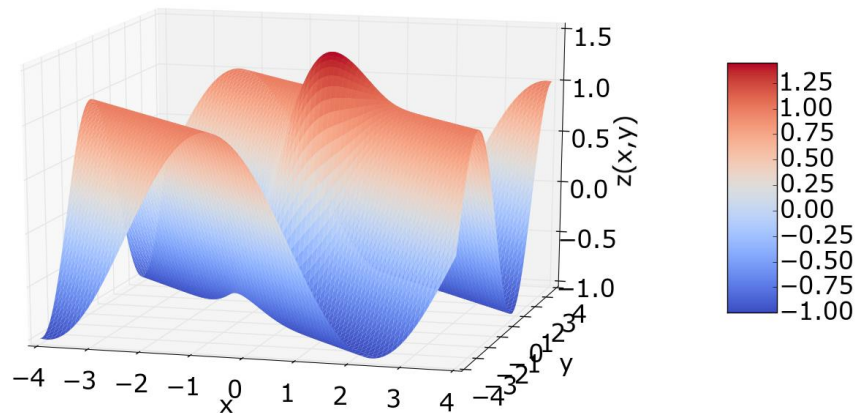


Fig. 5.7. Function $z(x, y)$ surface plot

3. Using Matlab and genetic algorithms find minimum of function $f(x, y) = 100 \cdot (x^2 - y)^2 + (1 - x)^2$ in the presence of the following constraints:

$$\begin{cases} x \cdot y + x - y + 1.5 \leq 0, \\ 10 - x \cdot y \leq 0, \\ 0 \leq x \leq 1, \\ 0 \leq y \leq 13. \end{cases}$$

Report contents.

- Goal of the work.
- Theoretical background.
- Found values of the minima und maximum of given functions with corresponding optimization processes.
- Conclusions about genetic algorithms effectiveness.

6. ADAPTIVE CONTROL SYSTEM TUNING BASED ON GENETIC ALGORITHMS

Goal of the work: studying genetic algorithms basics, mastering the skills of applying genetic algorithms for control system tuning.

Theoretical background. Genetic algorithms. Genetic algorithms theoretical basis is presented in laboratory work #5.

Let us consider the most important parameters of genetic algorithms, which can be specified in Matlab Optimization Tool.

Section 1 Problem Setup and Results.

Fitness function – handle of a fitness function that we want to minimize.

Number of variables – number of independent variables for the fitness function.

Subsection Constraints:

Linear inequalities of the form $\mathbf{Ax} \leq \mathbf{b}$ are specified by the matrix $\mathbf{A}$ and the vector $\mathbf{b}$.

Linear equalities of the form $\mathbf{A}_{eq}\mathbf{x} = \mathbf{b}_{eq}$ are specified by the matrix $\mathbf{A}_{eq}$ and the vector $\mathbf{b}_{eq}$.

Bounds are lower and upper bounds on the variables.

Nonlinear constraints is a function handle for nonlinear constraints function (see Laboratory work #5)

Integer variable indices is a vector whose entries give the components of $\mathbf{x}$ that are integer-valued (The use of integer variables imposes a number of restrictions on the use of the Matlab Optimization Tool, for example, there should be no constraints in the form of equations, both linear and nonlinear in the system).

Section 2 Options.

Population type (**Double vector**, **Bit string**, **Custom**) is variables types of the chromosome. **Double vector** is a vector of real numbers, **Bit string** is a string of binary codes, **Custom** is a user type.

Population size specifies how many individuals there are in each generation. If you set Population size to be a vector of length greater than 1, the algorithm creates multiple subpopulations. Each entry of the vector specifies the size of a subpopulation.

Creation function (**Uniform**, **Feasible population**, **Constraint dependent**, **Custom**) specifies the function that creates the initial population. **Uniform** creates a random initial population with a uniform distribution. **Feasible population** creates a random initial population that satisfies the bounds and linear con-

straints. **Constraint dependent** chooses **Uniform** if there are no constraints and **Feasible population** otherwise.

Initial population enables you to specify an initial population for the genetic algorithm. If you do not specify an initial population, the algorithm creates one using the **Creation function**. If you specify fewer than Population size individuals, than the **Creation function** creates the rest.

Initial scores enables you to specify scores for the initial population. If you do not specify Initial scores, the algorithm computes the scores using the fitness function.

Initial range specifies lower and upper bounds for the entries of the vectors in the initial population.

Fitness scaling (**Rank**, **Proportional**, **Top**, **Shift linear**, **Custom**) specifies the function that converts raw fitness scores returned by the fitness function to values in a range that is suitable for the selection function. **Rank** scales the raw scores based on the rank of each individual, rather than its score. The rank of an individual is its position in the sorted scores. **Proportional** makes the expectation proportional to the raw fitness score. **Top** scales the individuals with the highest fitness values equally. If you select this option, you can specify Quantity, the number of fittest individuals that produce offspring. Each of these individuals has an equal probability of reproducing. The rest have probability 0 of reproducing. **Shift linear** scales the raw scores so that the expectation of the fittest individual is equal to a constant, which you can specify as Maximum survival rate, multiplied by the average score. **Custom** allows to set user defined scaling function.

Selection function (**Stochastic uniform**, **Remainder**, **Uniform**, **Roulette**, **Tournament**, **Custom**) chooses parents for the next generation based on their scaled values from the fitness scaling function. **Stochastic uniform** lays out a line in which each parent corresponds to a section of the line of length proportional to its expectation. The algorithm moves along the line in steps of equal size, one step for each parent. At each step, the algorithm allocates a parent from the section it lands on. **Remainder** assigns parents deterministically from the integer part of each individual's scaled value and then uses roulette selection on the remaining fractional part. **Uniform** select parents at random from a uniform distribution using the expectations and number of parents. This results in an undirected search. **Roulette** simulates a roulette wheel with the area of each segment proportional to its expectation. The algorithm then uses a random number to select one of the sections with a probability equal to its area. **Tournament** selects each parent by choosing individuals at random, the number of which you can specify by Tournament size, and then choosing the best individual out of that set to be a parent. **Custom** allows to set user defined selection function.

Reproduction options determine how the genetic algorithm creates children at each new generation. **Elite count** specifies the number of individuals that are guaranteed to survive to the next generation. **Crossover fraction** specifies the fraction of the next generation that crossover produces. Mutation produces the remaining individuals in the next generation.

Mutation function (**Constraint dependent**, **Gaussian**, **Uniform**, **Adaptive feasible**, **Custom**) makes small random changes in the individuals in the population, which provide genetic diversity and enable the genetic algorithm to search a broader space. **Gaussian** adds a random number to each vector entry of an individual. This random number is taken from a Gaussian distribution centered on zero. The standard deviation of this distribution can be controlled with two parameters. The **Scale** parameter determines the standard deviation at the first generation. The **Shrink** parameter controls how standard deviation shrinks as generations go by. **Uniform** is a two-step process. First, the algorithm selects a fraction of the vector entries of an individual for mutation, where each entry has the same probability as the mutation rate of being mutated. In the second step, the algorithm replaces each selected entry by a random number selected uniformly from the range for that entry. **Adaptive feasible** randomly generates directions that are adaptive with respect to the last successful or unsuccessful generation. A step length is chosen along each direction so that linear constraints and bounds are satisfied. **Constraint dependent** chooses **Gaussian** if there are no constraints and **Adaptive feasible** otherwise. **Custom** allows to set user defined mutation function.

Crossover function (**Scattered**, **Single point**, **Two point**, **Intermediate**, **Heuristic**, **Arithmetic**, **Custom**) combines two individuals, or parents, to form a new individual, or child, for the next generation. **Scattered** creates a random binary vector. It then selects the genes where the vector is a 1 from the first parent, and the genes where the vector is a 0 from the second parent, and combines the genes to form the child. **Single point** chooses a random integer n between 1 and **Number of variables**, and selects the vector entries numbered less than or equal to n from the first parent, selects genes numbered greater than n from the second parent, and concatenates these entries to form the child. **Two point** selects two random integers m and n between 1 and **Number of variables**. The algorithm selects genes numbered less than or equal to m from the first parent, selects genes numbered from $m+1$ to n from the second parent, and selects genes numbered greater than n from the first parent. The algorithm then concatenates these genes to form a single chromosome. **Intermediate** calculates child genes with following formula $child_1 = parent_1 + rand \cdot Ratio \cdot (parent_2 - parent_1)$, where $rand$ is a random number and $Ratio$ parameter is set with **Ratio** field. **Heuristic** creates children that randomly lie on the line containing the two parents, a small distance away

from the parent with the better fitness value, in the direction away from the parent with the worse fitness value. **Arithmetic** creates children that are a random arithmetic mean of two parents, uniformly on the line between the parents. **Custom** allows to set user defined crossover function.

Migration is the movement of individuals between subpopulations, which the algorithm creates if you set Population size to be a vector of length greater than 1. Every so often, the best individuals from one subpopulation replace the worst individuals in another subpopulation.

Direction (Forward, Both) specifies the direction in which migration can take place.

Fraction controls how many individuals move between subpopulations.

Example of using of genetic algorithms in Matlab for direct adaptive control system tuning. Let us consider an adaptive control system for double-mass elastic plant from laboratory work #4. In the previous work, the weight matrix **P** was calculated using the Lyapunov equation (1.5) and arbitrary symmetric positive definite matrix **Q**. It was also shown that choice of the **Q** matrix has a great influence on the system step response quality. Let us now use genetic algorithms to find coefficients of **P** matrix. Because the system has only one input and only one non-zero coefficient in the **b** matrix, only one line of **P** matrix takes part in adaptive control adjustment algorithms (1.4). Therefore, in our example, only 4 coefficients of **P** matrix must be found. Let us form fitness function in the following way:

1. Obtain reference model step response with fixed-step integration method and step size Δt .

2. Obtain adaptive system step responses with decreased and increased in 4 times in all combinations all unknown system parameters with the same integration step size Δt .

3. Calculate the sum of absolute values of differences between adaptive system and reference model step responses for each integration point

$$s_j = \sum_{i=1}^l |x_{j_i} - x_{m_i}|, \text{ where } l \text{ is the number of integration points.}$$

4. As a result of fitness function we take the largest value from calculated values in step 3 $f = \max s_j$.

For our example as unknown parameters we will take second mass moment of inertia and link stiffness. In this case fitness function can be written in the following form:

```
function fit = test_fitness(x)
P(4,1) = 20.7754524965707;
P(4,2) = 56.5826276830420;
P(4,3) = 4.53378207133062;
P(4,4) = 3.03918804755375;
```

```

assignin('base', 'P', P);
I2 = 0.1;
p = 0.8;
I2_nom = I2;
p_nom = p;
assignin('base', 'I2', I2);
assignin('base', 'p', p);
sim('elasticSystem');
youtm = yout;
P = zeros(4);
P(4,1) = x(1) / 10;
P(4,2) = x(2) / 10;
P(4,3) = x(3) / 10;
P(4,4) = x(4) / 10;
assignin('base', 'P', P);
result = [];
fit = 0;

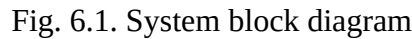
params = [[I2_nom / 4, p_nom    ];
          [I2_nom * 4, p_nom    ];
          [I2_nom,      p_nom / 4];
          [I2_nom * 4, p_nom / 4];
          [I2_nom / 4, p_nom / 4]];

try
    for i = 1:length(params)
        I2 = params(i, 1);
        p  = params(i, 2);
        assignin('base', 'I2', I2);
        assignin('base', 'p', p);
        sum = 0;
        sim('elasticSystem');
        for j = 1:5:length(yout)
            sum = sum + abs(yout(j,1) - youtm(j,1));
        end
        result = [result (sum / 1000)];

        display(sum / 1000);
    end
    m = max(result);
    fit = m;
catch
    fit = 1000000000000000;
end
display(P)
display(x)
display(fit)

```

where for reference model step response calculation matrix $\mathbf{P}$ calculated using Lyapunov function is used. In order to obtain system state vector in **yout** variable we have to add output ports to the system block diagram. System block diagram is shown in Fig. 6.1.



Let us now consider step responses of adaptive control system with parametric adjustment calculated using genetic algorithms in comparison with system calculated using Lyapunov equation and matrix $\mathbf{Q}_3$, that has shown the best results in laboratory work #4. Systems step responses with reduced in 4 times second mass moment of inertia is shown in Fig. 6.3. Systems step responses with increased in 4 times second mass moment of inertia is shown in Fig. 6.4. Systems step responses with reduced in 4 times link stiffness is shown in Fig. 6.5. Systems step responses with reduced in 4 times second mass moment of inertia and link stiffness is shown in Fig. 6.6.

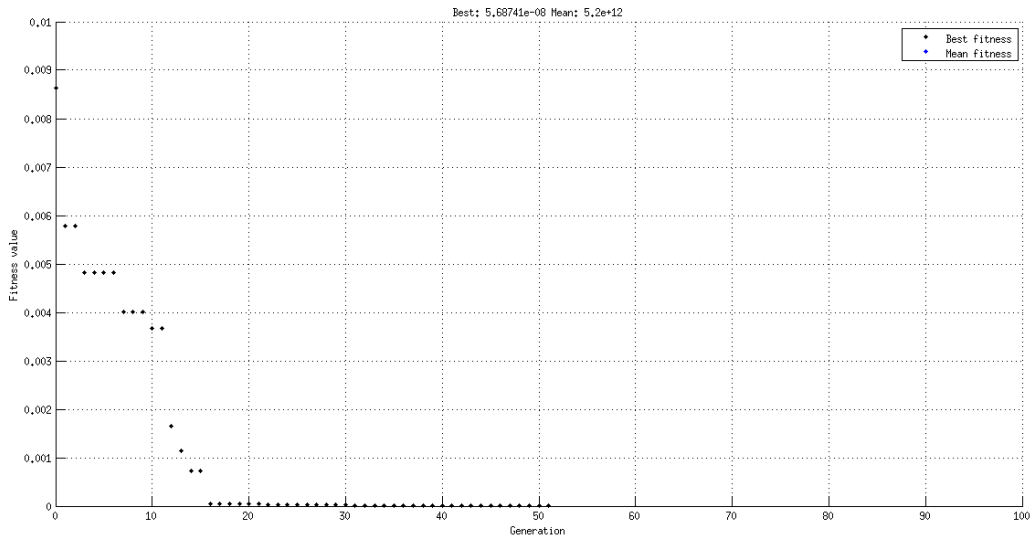


Fig. 6.2. Optimization process

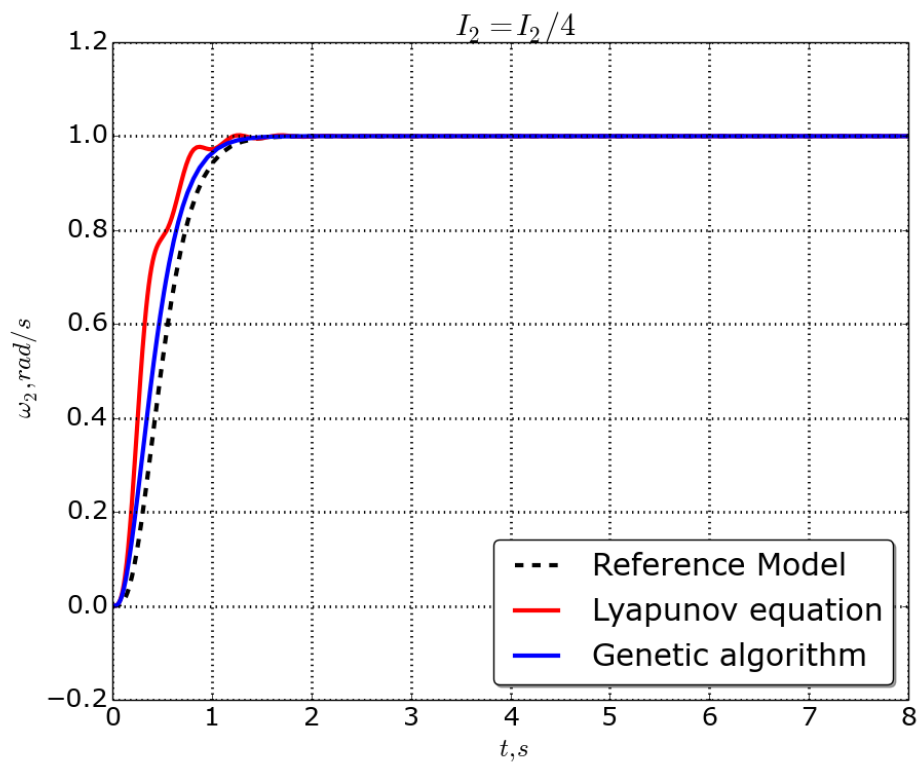


Fig. 6.3. Systems step responses with reduced in 4 times second mass moment of inertia

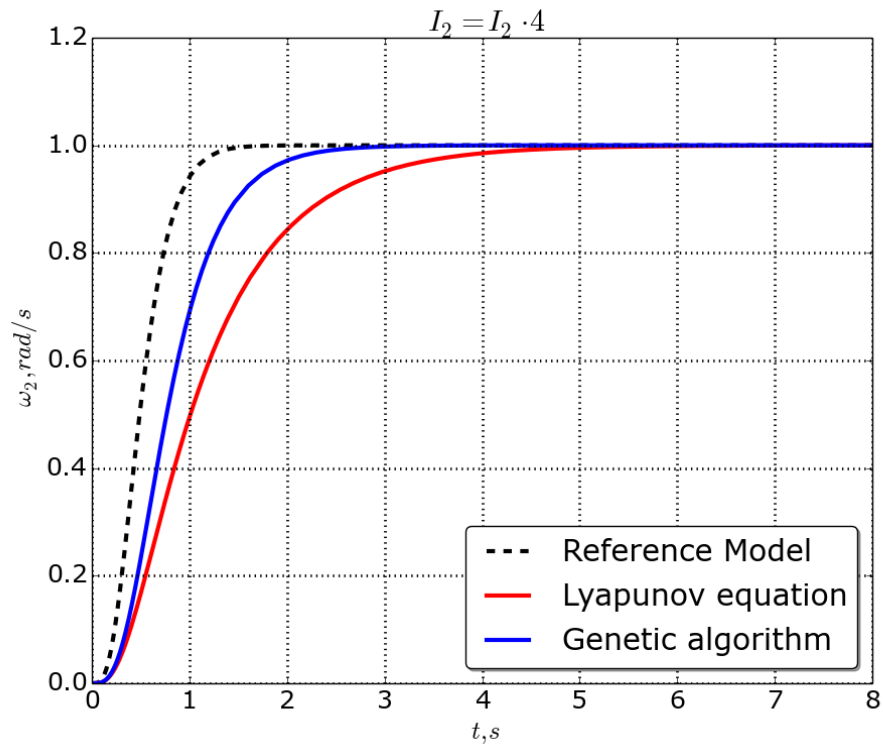


Fig. 6.4. Systems step responses with increased in 4 times second mass moment of inertia

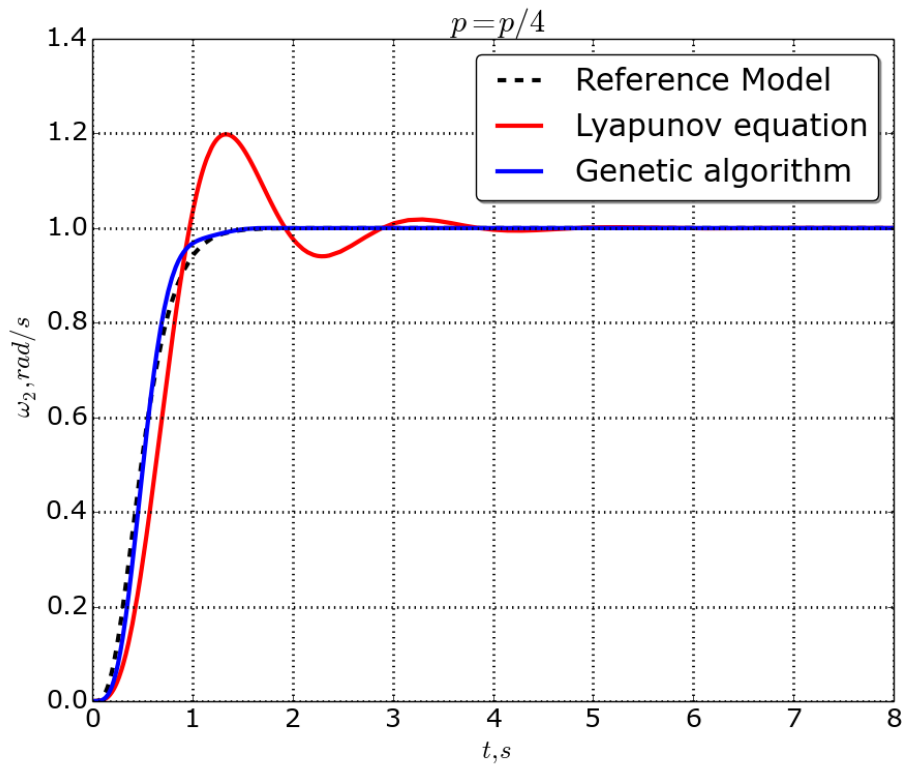


Fig. 6.5. Systems step responses with reduced in 4 times link stiffness

Systems step responses with reduced in 4 times link stiffness and increased in 4 times second mass moment of inertia is shown in Fig. 6.7.

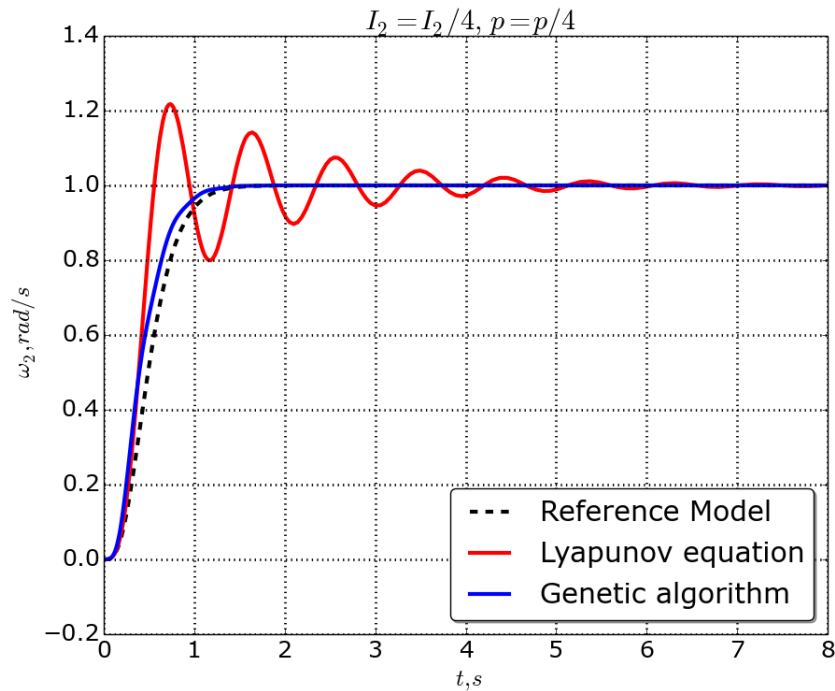


Fig. 6.6. Systems step responses with reduced in 4 times second mass moment of inertia and link stiffness

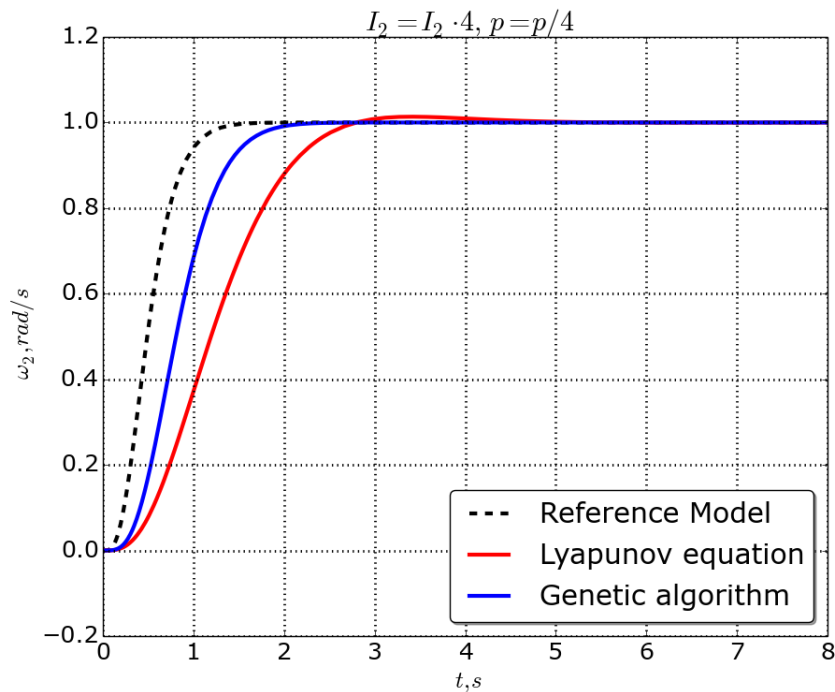


Fig. 6.7. Systems step responses with reduced in 4 times link stiffness and increased in 4 times second mass moment of inertia

As can be seen from step responses, the use of genetic algorithms in adaptive control system calculation allows to improve system step responses quality.

Laboratory work program.

1. Using genetic algorithms and Matlab Optimization Tool calculate parameters of direct adaptive control system with parametric adjustment for double-mass elastic plant.
2. Make a comparison of influence of changing 3 different genetic algorithm parameters on the optimization process.

Variants of double-mass plant parameters are the same as in laboratory work #4

Report contents.

- Goal of the work.
- Theoretical background.
- Adaptive control system calculation using genetic algorithms.
- Comparison with results of laboratory work #4.
- Comparison of genetic algorithm parameters influence on the optimization process.
- Conclusions about genetic algorithms effectiveness for adaptive control system calculation.

7. LÜDERS-NARENDRA ADAPTIVE OBSERVER AND IDENTIFIER

Goal of the work: studying adaptive observers and identifiers basics, mastering the skills of applying adaptive observers and identifiers for system state vector estimation.

Theoretical background. Lüders-Narendra adaptive observer and identifier.

In order to implement several control systems, such as full state feedback control or adaptive control with parameter adjustment full state vector is required. However, in practice, it is not always possible to measure complete system state vector. Further, measurement noise is invariably present in the system, and therefore the complete state cannot be reconstructed by taking an adequate number of derivatives of the output. Time-invariant state observer can be applied if the plant parameters are known, but in the case of parametric uncertainty, it can lead to system op-

eration deterioration. A possible solution for complete state vector estimation in presents of parametric uncertainty is Lüders-Narendra adaptive observer and identifier. Let us consider linear time-invariant single input single output n -th order system with the following form:

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{b}u(t) \\ y = \mathbf{c}\mathbf{x} \end{cases}, \quad (7.1)$$

this system can be represented in a form of n -th order transfer function, therefore only $2n$ parameters estimation is required for system identification. If and only if the system (7.1) is completely observable, then it can always be represented in the following form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} a_1 & 1 & \dots & 1 \\ a_2 & & & \\ \vdots & & \Lambda & \\ a_n & & & \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} u, \quad (7.2)$$

$$y = [1 \ 0 \ \dots \ 0] \mathbf{x}$$

where Λ is an $(n-1) \times (n-1)$ diagonal matrix with arbitrary but known constant and negative diagonal elements $-\lambda_i (i=2 \dots n)$, and $a = (a_1, a_2, \dots, a_n)^T$ and $b = (b_1, b_2, \dots, b_n)^T$ are the $2n$ unknown parameters to be identified. System (7.2) transfer function can be represented in the following form:

$$T(s) = \frac{b_1 + b_2 \frac{1}{s + \lambda_2} + \dots + b_n \frac{1}{s + \lambda_n}}{s - a_1 - a_2 \frac{1}{s + \lambda_2} - \dots - a_n \frac{1}{s + \lambda_n}}. \quad (7.3)$$

Let us define auxiliary signals in the following form:

$$\begin{aligned} \dot{v}_i + \lambda_i v_i &= x_i, i = 2, \dots, n \\ \dot{s}_i + \lambda_i s_i &= u, i = 2, \dots, n \end{aligned} \quad (7.4)$$

and estimating algorithms for unknown $2n$ parameters have the following form:

$$\begin{aligned} \dot{\alpha}_1 &= -c_1 x_1 (\hat{x}_1 - x_1) & \dot{\beta}_1 &= -d_1 u (\hat{x}_1 - x_1) \\ \dot{\alpha}_2 &= -c_2 v_2 (\hat{x}_1 - x_1) & \dot{\beta}_2 &= -d_2 s_2 (\hat{x}_1 - x_1) \\ &\vdots & &\vdots \\ \dot{\alpha}_n &= -c_n v_n (\hat{x}_1 - x_1) & \dot{\beta}_n &= -d_n s_n (\hat{x}_1 - x_1) \end{aligned}, \quad (7.5)$$

where c_i and d_i are arbitrary but positive constants (adaptive gains). In that case adaptive observer equations have the following form:

$$\begin{aligned}
\dot{\hat{x}}_1 &= \alpha_1 x_1 + \hat{x}_2 + \dots + \hat{x}_n + \beta_1 u - \lambda_1 (\hat{x}_1 - x_1) \\
\dot{\hat{x}}_2 &= \alpha_2 x_1 - \lambda_2 \hat{x}_2 + \beta_2 u + (c_2 v_2^2 + d_2 s_2^2)(\hat{x}_1 - x_1), \\
&\vdots \\
\dot{\hat{x}}_n &= \alpha_n x_1 - \lambda_n \hat{x}_n + \beta_n u + (c_n v_n^2 + d_n s_n^2)(\hat{x}_1 - x_1)
\end{aligned} \tag{7.6}$$

where λ_1 is arbitrary but positive constant.

Example of Lüders-Narendra adaptive observer and identifier calculation. Let us consider linear time-invariant 2nd order plant from laboratory work #3 with following parameters:

$$\mathbf{A} = \begin{bmatrix} 0 & a_1 \\ -a_2 & -a_2 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ a_2 \end{bmatrix} \text{ and } \mathbf{c} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \text{ where } a_1 = 3, a_2 = 2.$$

Plant state space representation does not correspond to the representation (7.2), therefore it is required to transform system in order to use adaptive observer. At first we obtain a plant transfer function using Matlab `ss2tf` function, which has the following form $[\mathbf{NUM}, \mathbf{DEN}] = \mathbf{ss2tf}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$, where $\mathbf{NUM}$ is a transfer function numerator coefficients vector, $\mathbf{DEN}$ is a transfer function denominator coefficients vector and $\mathbf{D}$ is feedthrough matrix (in our example it is equal to 0). In our example transfer function coefficients has following values: $\mathbf{NUM} = [0, 0, 6]$, $\mathbf{DEN} = [1, 2, 6]$, which corresponds to the following transfer function:

$$T(s) = \frac{6}{s^2 + 2s + 6}, \tag{7.7}$$

Transfer function of the 2nd order system in the form (7.2) according to (7.3) can be written using the following form:

$$T_o(s) = \frac{b_1 + \frac{b_2}{s + \lambda_2}}{s - a_1 - \frac{a_2}{s + \lambda_2}} = \frac{b_1 s + b_1 \lambda_2 + b_2}{s^2 + (\lambda_2 - a_1)s - a_1 \lambda_2 - a_2}. \tag{7.8}$$

Equating corresponding coefficients with equal powers of s we can following equations for system in the form (7.2) with respect to λ_2 value:

$$\mathbf{A}_o = \begin{bmatrix} \lambda_2 - 2 & 1 \\ -\lambda_2^2 + 2\lambda_2 - 6 & -\lambda_2 \end{bmatrix}, \mathbf{b}_o = \begin{bmatrix} 0 \\ 6 \end{bmatrix}. \tag{7.9}$$

Let us take the following adaptive observer parameters values for our example: $\lambda_1 = 100$, $\lambda_2 = 20$, $c_1 = 100$, $c_2 = 100$, $d_1 = 100$ and $d_2 = 100$. System block diagram is presented in Fig. 7.1. Adaptive Observer subsystem block diagram is presented in Fig. 7.2.

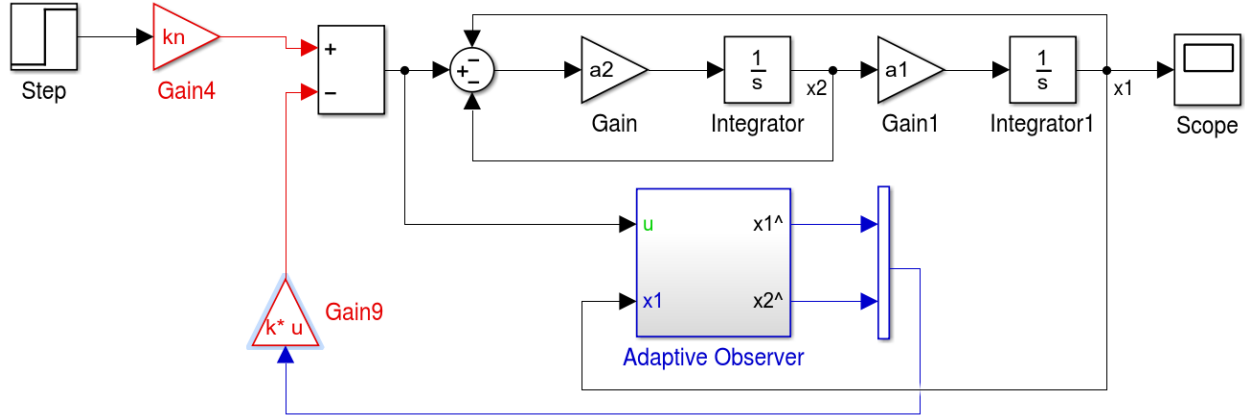


Fig. 7.1. System block diagram

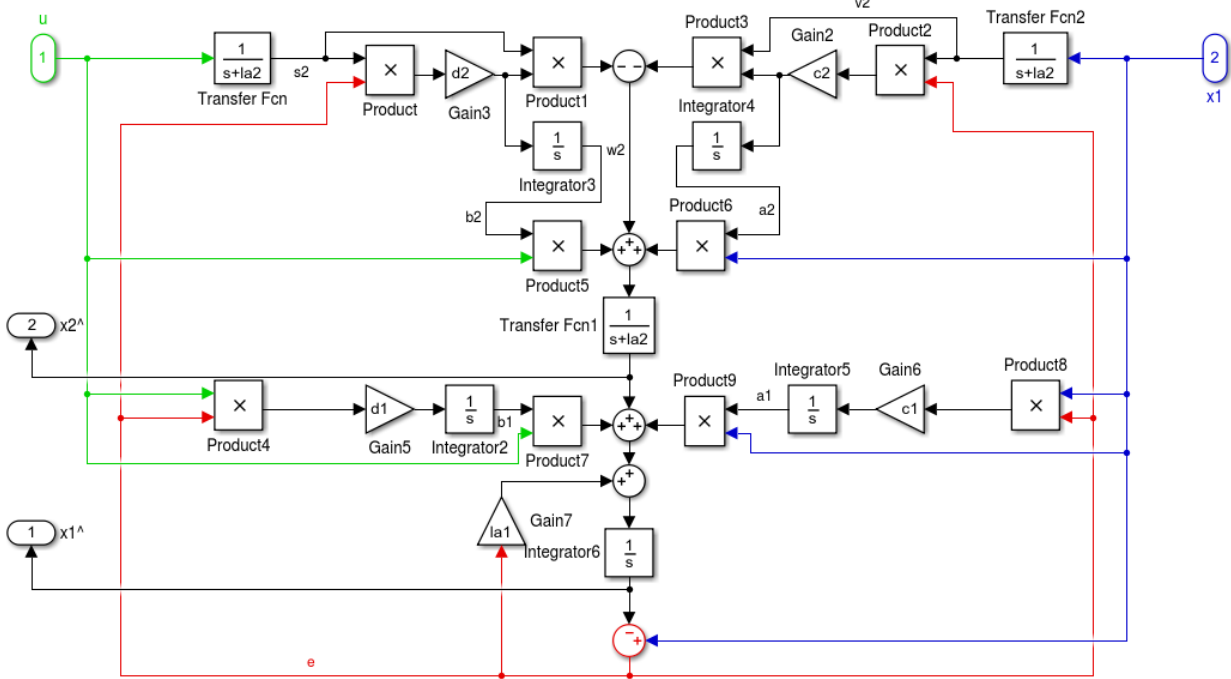


Fig. 7.2. Adaptive Observer subsystem block diagram

As identified plant parameters initial values we take parameters of matrix $\mathbf{A}_o$ calculated with plant nominal parameters. We set these values as initial values of corresponding integrators. Full state feedback parameters for system (7.9) has following values: $\mathbf{k}=[23.375 \quad 1.1667]$, $k_n=3.375$. Step responses of the system with adaptive observer and full state feedback control with parameters a_1 and a_2 changing are presented in Fig. 7.3. As can be seen from step responses, use of adaptive observer allows to estimate the complete system state vector.

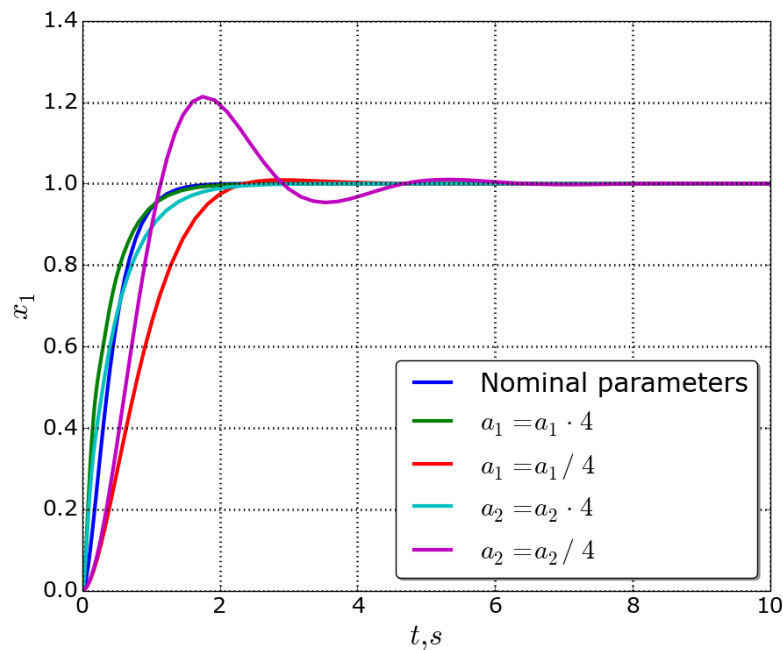


Fig. 7.3. Step responses of the system with adaptive observer and full state feedback control with parameters a_1 and a_2 changing

Laboratory work program.

1. Investigate system (3.5) with given parameters a_1 and a_2 .
2. Calculate full state feedback control (3.1) and investigate fully measured system with nominal parameters and increased and decreased parameters a_1 and a_2 in 4 times.
3. Calculate adaptive observer (7.4-7.6) and investigate system with full state feedback control and adaptive observer with nominal parameters and increased and decreased parameters a_1 and a_2 in 4 times.
4. Compare calculated values of matrix $\mathbf{A}_o$ and identified ones with adaptive observer and identifier with plant parameters changing.

Variants of plant parameters are the same as in laboratory work #3

Report contents.

- Goal of the work.
- Theoretical background.
- System calculation in the form (7.2).
- Full state feedback control calculation for the system in the form (7.2).

- Lüders-Narendra adaptive observer and identifier calculation.
- System with full state feedback and adaptive observer investigation in presents of parameters changing.
- Comparison of calculated values of matrix $\mathbf{A}_o$ and identified ones.
- Conclusions about Lüders-Narendra adaptive observer and identifier effectiveness for the state vector estimation and plant parameters identification.

8. SLIDING MODE STATE OBSERVER

Goal of the work: studying sliding mode state observers, mastering the skills of applying sliding mode state observers for system state vector estimation.

Theoretical background. Sliding mode state observer. In order to estimate complete system state vector in presence of parametric uncertainty it is also possible to apply sliding mode state observer. Let us consider linear time-invariant system of the n -th order:

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{A}\mathbf{x} + \mathbf{b}u \\ \mathbf{y} &= \mathbf{C}\mathbf{x}\end{aligned}\quad (8.1)$$

where $\mathbf{x} \in R^n$, $\mathbf{y} \in R^m$. If system (8.1) is observable and $\mathbf{C}$ has full rank then sliding mode state observer has the following form:

$$\dot{\hat{\mathbf{x}}} = \mathbf{A}\hat{\mathbf{x}} + \mathbf{b}u + \mathbf{L}_s \text{sgn}(\mathbf{y} - \hat{\mathbf{y}}), \quad (8.2)$$

where sgn function is defined as columnwise

$$\text{sgn} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} \text{sgn}(x_1) \\ \text{sgn}(x_2) \\ \vdots \\ \text{sgn}(x_n) \end{bmatrix}, \text{sgn}(x) = \begin{cases} 1, x > 0 \\ -1, x < 0 \end{cases}$$

and $\mathbf{L}_s \in R^{n \times m}$ is an observer gain matrix as to be chosen so as to satisfy the convergence of $\hat{\mathbf{x}}$ to $\mathbf{x}$.

Let us apply static transformation to obtain directly the available m outputs of the system as a part of its n dimensional state vector using the following nonsingular transformation matrix

$$\mathbf{T} = \begin{bmatrix} \mathbf{C} \\ \mathbf{R} \end{bmatrix}, \quad (8.3)$$

where $\mathbf{R} \in R^{(n-m) \times n}$ is arbitrary provided that $\mathbf{T}$ is invertible and defining $\mathbf{x}_1 = \mathbf{R}\mathbf{x}$, where $\mathbf{x}_1 \in R^{n-m}$. The following new system representation is obtained

$$\begin{aligned}\dot{\mathbf{y}} &= \mathbf{A}_{11}\mathbf{y} + \mathbf{A}_{12}\mathbf{x}_1 + \mathbf{b}_1u \\ \dot{\mathbf{x}}_1 &= \mathbf{A}_{21}\mathbf{y} + \mathbf{A}_{22}\mathbf{x}_1 + \mathbf{b}_2u\end{aligned}\quad (8.4)$$

where $\mathbf{TAT}^{-1} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}$ and $\mathbf{Tb} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$.

Sliding mode state observer equations in this case have the following form

$$\begin{aligned}\dot{\hat{\mathbf{y}}} &= \mathbf{A}_{11}\hat{\mathbf{y}} + \mathbf{A}_{12}\hat{\mathbf{x}}_1 + \mathbf{b}_1u + \mathbf{L}_1 \text{sgn} \mathbf{e}_y \\ \dot{\hat{\mathbf{x}}}_1 &= \mathbf{A}_{21}\hat{\mathbf{y}} + \mathbf{A}_{22}\hat{\mathbf{x}}_1 + \mathbf{b}_2u + \mathbf{L}_2 \text{sgn} \mathbf{e}_y\end{aligned}\quad (8.5)$$

where $\mathbf{e}_y = \mathbf{y} - \hat{\mathbf{y}}$ is an estimation error, $\mathbf{L}_1 \in R^{m \times m}$, $\mathbf{L}_2 \in R^{(n-m) \times m}$ are the parameters of the observer.

Sliding mode state observer (8.2) reconstructs the system (8.1) state vector asymptotically choosing

$$\mathbf{L}_s = \mathbf{T}^{-1} \begin{bmatrix} \mathbf{L}_1 \\ \mathbf{L}_2 \mathbf{L}_1 \end{bmatrix},$$

$\mathbf{L}_1 \in R^{m \times m}$ is a positive definite diagonal matrix whose each diagonal element is greater than $\|\mathbf{A}_{11}\mathbf{e}_y + \mathbf{A}_{12}\mathbf{e}_{x1}\|_\infty$, $\mathbf{L}_2 \in R^{(n-m) \times m}$ is the matrix which places the eigenvalues of $(\mathbf{A}_{22} - \mathbf{L}_2\mathbf{A}_{12})$ at desired and $\mathbf{e}_{x1} = \mathbf{x}_1 - \hat{\mathbf{x}}_1$.

Example of sliding mode state observer calculation. Adaptive control system from laboratory work #6 uses complete state vector of the system, but it is not always possible to measure complete state vector. We assume that there are a second mass rotation velocity and DC motor armature current sensors in the system. In order to estimate complete state vector of the system let us apply full-order Luenberger state observer from laboratory work #3 and also sliding mode state observer. State observer dynamics should be faster than dynamics of the adaptive control system, thus we take the following observer poles vector $\mathbf{p}_o = [-450 \quad -450.0001 \quad -449.9999 \quad -450]$. Since there are 2 sensors in the system,

its output matrix has the following values $\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. We cannot use

in this case Matlab “acker” function in order to calculate observer gain matrix because $\mathbf{C}$ matrix has 2 rows, but we can use similar function “place” and the following command $\mathbf{L}_o = \text{place}(\mathbf{A}', \mathbf{C}', \mathbf{p}_o)'$, where $\mathbf{p}_o$ is desired poles vector. “Place” function might fail to place all poles at one point, that is why poles values are not the same, but close. Thus Luenberger observer gain matrix has the following values:

$$\mathbf{L}_o = \begin{bmatrix} 899.999 & -0.08 \\ 20248 & -7.2018 \\ -1800 & -2004.8 \\ 199.999 & 700 \end{bmatrix}.$$

Let us now calculate sliding mode state observer parameters. Since only the first and fourth state variables can be measured the transformation matrix of the not measuring state variables has the following form $\mathbf{R} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$. In this case system transformation matrix according to (8.3) has the following values:

$$\mathbf{T} = \begin{bmatrix} \mathbf{C} \\ \mathbf{R} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

In this case according to (8.4) we obtain the following matrices of system (8.5):

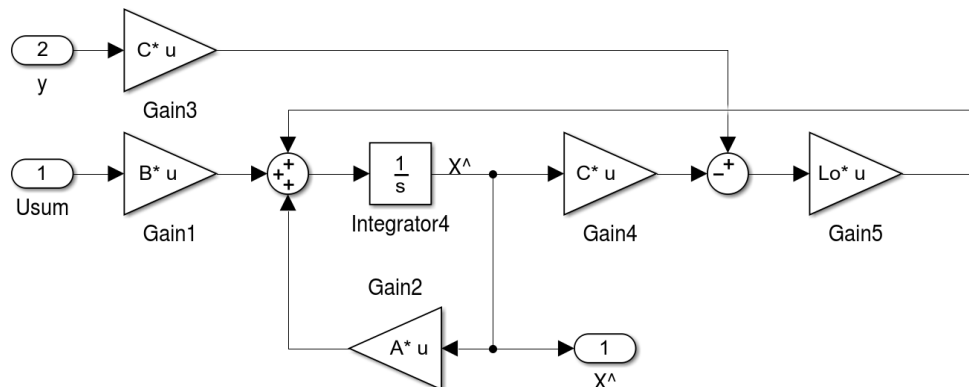
$$\mathbf{A}_{11} = \begin{bmatrix} 0 & 0 \\ 0 & -\frac{R}{L} \end{bmatrix}, \mathbf{A}_{12} = \begin{bmatrix} \frac{1}{J_2} & 0 \\ 0 & -\frac{k_e}{L} \end{bmatrix}, \mathbf{A}_{21} = \begin{bmatrix} -p & 0 \\ 0 & \frac{k_m}{J_1} \end{bmatrix}, \mathbf{A}_{22} = \begin{bmatrix} 0 & p \\ -\frac{1}{J_1} & 0 \end{bmatrix}, \mathbf{b}_1 = \begin{bmatrix} 0 \\ \frac{1}{L} \end{bmatrix},$$

$$\mathbf{b}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

For this example we choose first observer parameters matrix as the following diagonal positive definite matrix $\mathbf{L}_1 = \text{diag}[60 \ 60]$, and $\mathbf{p}_s = [-450 \ -450]$ as desired sliding mode state observer poles vector. Again we cannot use Matlab “acker” function, so we use “place” function and the following Matlab command `L2 = place(A22, A12, ps)`, where **A22** and **A11** are corresponding matrices from (8.5) and **ps** is desired observer poles vector, in order to calculate second observer parameters matrix. Required sliding mode state observer (8.2) gain matrix has the following values:

$$\mathbf{L}_s = \begin{bmatrix} 100 & 0 \\ 4500 & 8 \\ 20 & -450 \\ 0 & 100 \end{bmatrix}.$$

Luenberger observer subsystem block diagram is shown in Fig. 8.2 and sliding mode state observer subsystem block diagram is presented in Fig. 8.3.



54

In order to reduce simulation time in Matlab/Simulink, the sgn function was implemented as saturation block with limits $\begin{bmatrix} 1 & -1 \end{bmatrix}$ and high gain value.

System step responses with state observer and fourfold plant parameters changes are presented in Fig. 8.4-8.11.

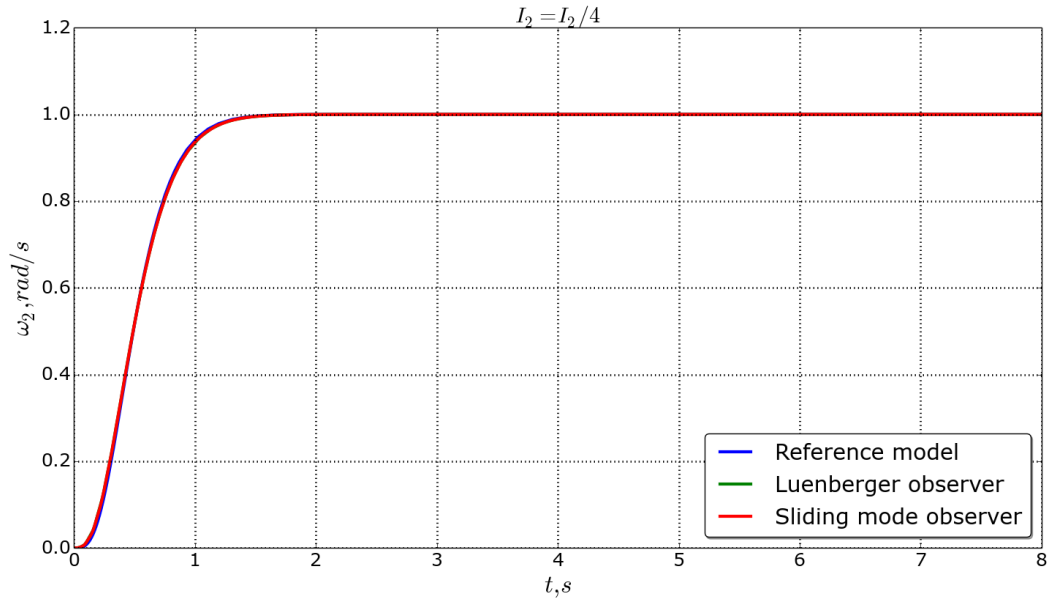


Fig. 8.4. Systems step responses with reduced second mass moment of inertia

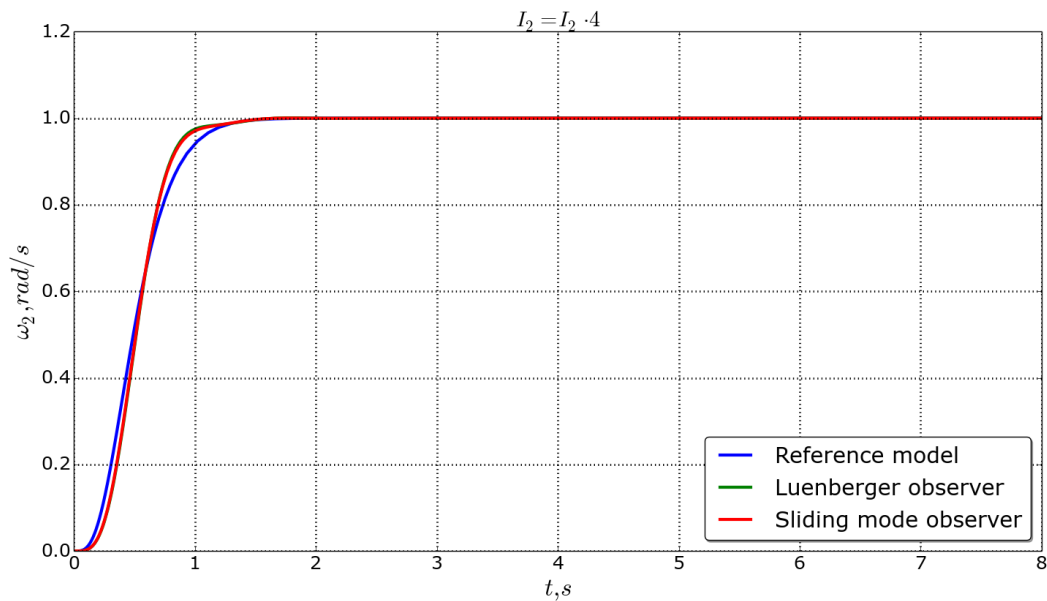


Fig. 8.5. Systems step responses with increased second mass moment of inertia

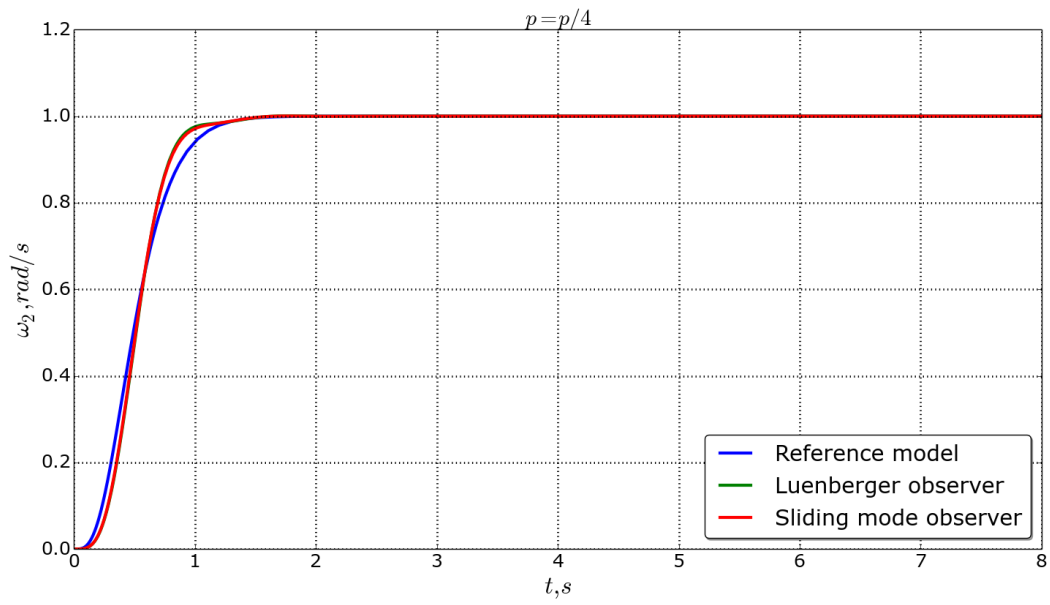


Fig. 8.6. Systems step responses with reduced link stiffness

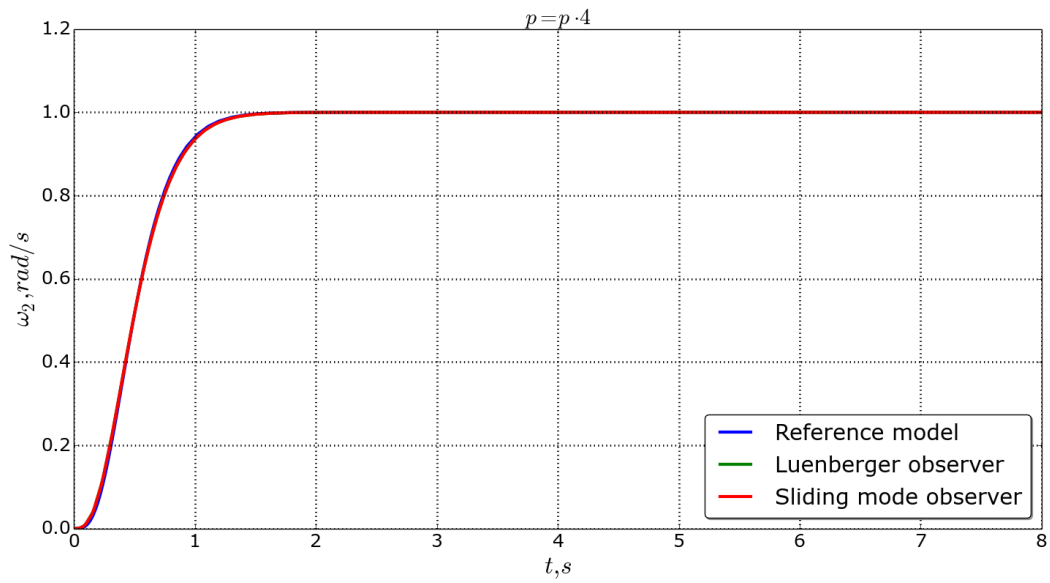


Fig. 8.7. Systems step responses with increased link stiffness

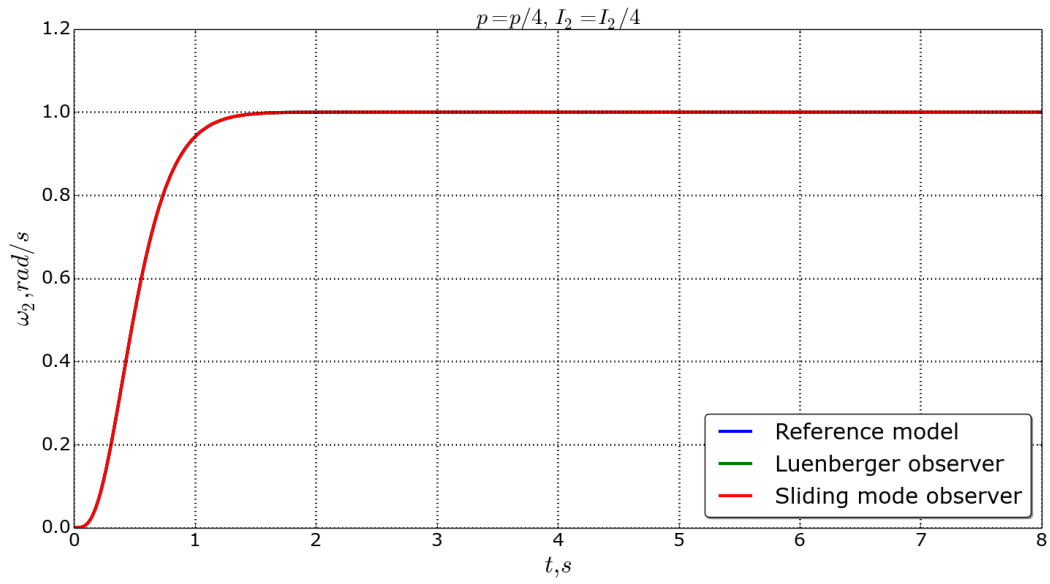


Fig. 8.8. Systems step responses with reduced link stiffness and reduced second mass moment of inertia

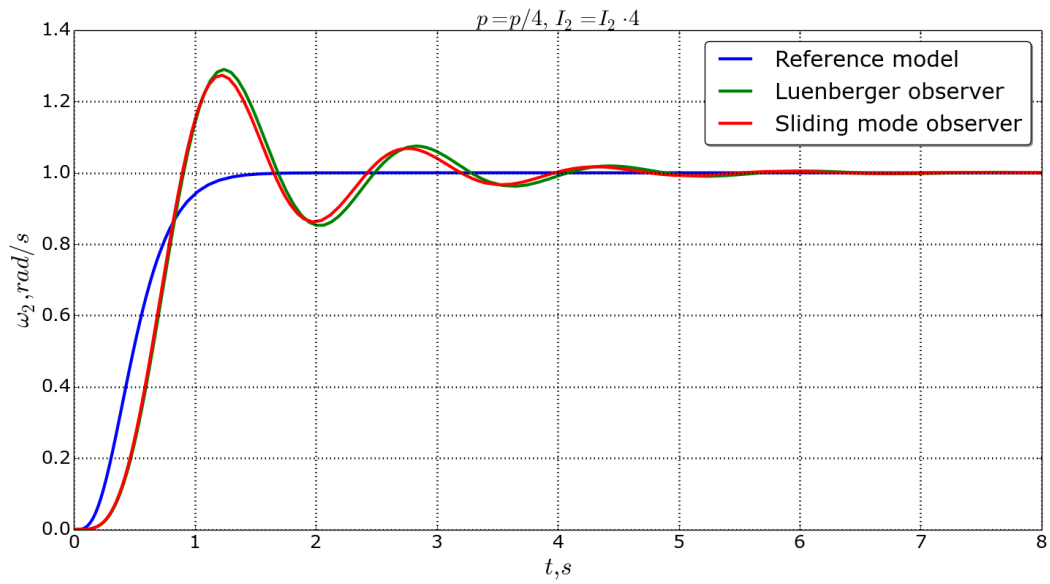


Fig. 8.9. Systems step responses with reduced link stiffness and increased second mass moment of inertia

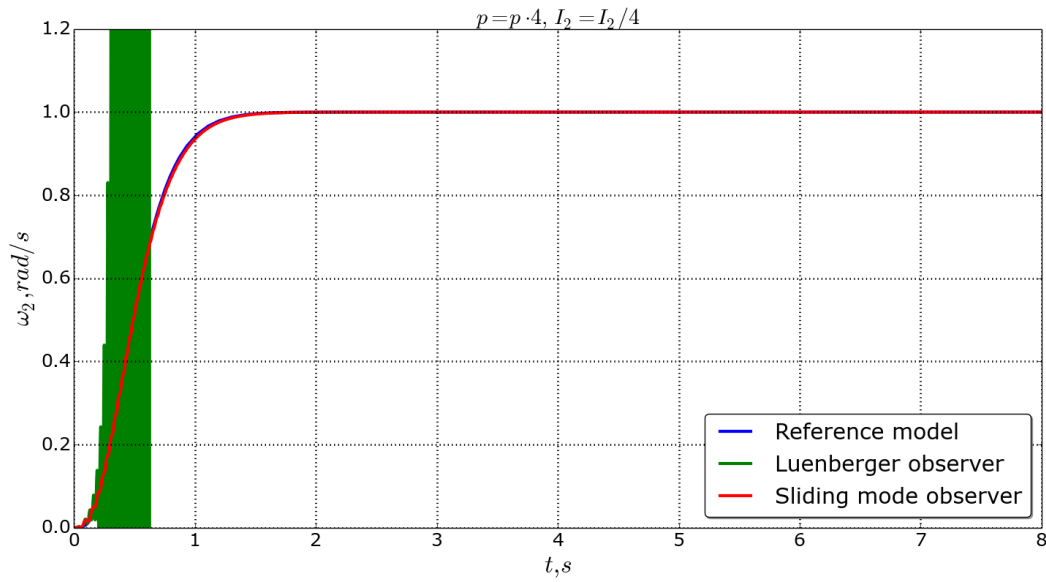


Fig. 8.10. Systems step responses with increased link stiffness and reduced second mass moment of inertia

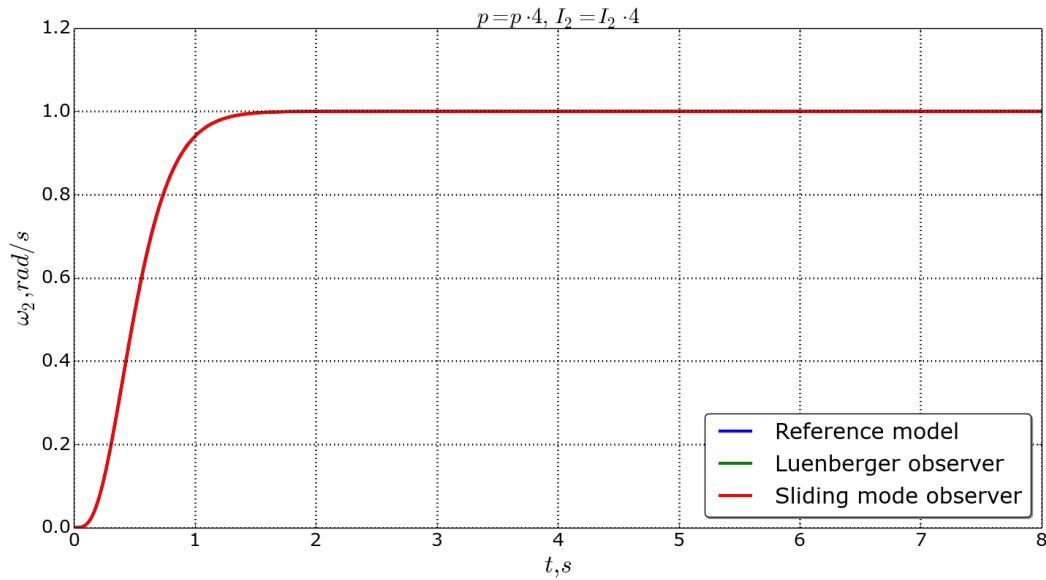


Fig. 8.11. Systems step responses with increased link stiffness and increased second mass moment of inertia

As can be seen from obtained step responses, the use of Luenberger state observer with considering system gives close results to the results of the same system with sliding mode state observer, but in the system with increased link stiffness and reduced second mass moment of inertia the use of Luenberger state observer can lead to increasing oscillations while the same system with sliding mode state observer has step response close to reference model step response.

Laboratory work program.

1. Calculate Luenberger state observer for double mass system from laboratory work #6 with second mass rotation velocity and DC motor armature current sensors.
2. Investigate obtained system with fourfold plant parameters changes.
3. Calculate sliding mode state observer for double mass system from laboratory work #6 with second mass rotation velocity and DC motor armature current sensors.
4. Investigate obtained system with fourfold plant parameters changes.
5. Calculate Luenberger state observer for double mass system from laboratory work #6 with second mass rotation velocity and first mass rotation velocity sensors.
6. Investigate obtained system with fourfold plant parameters changes.
7. Calculate sliding mode state observer for double mass system from laboratory work #6 with second mass rotation velocity and first mass rotation velocity sensors.
8. Investigate obtained system with fourfold plant parameters changes.

Variants of double-mass plant parameters are the same as in laboratory work #4

Report contents.

- Goal of the work.
- Theoretical background.
- Luenberger state observers calculations for double mass system.
- Double mass system investigation with Luenberger state observers.
- Sliding mode state observer calculation for double mass system.
- Double mass system investigation with sliding mode state observers.
- Conclusions about Luenberger state observer and sliding mode state observer effectiveness for adaptive control system for double mass systems.

TEST QUESTIONS

1. What is used in direct adaptive control systems in order to get desired system transient response?
2. What is the formula for control action for direct adaptive control system with parametric adjustment?
3. How can we calculate $\mathbf{P}$ matrix for differential adjustment equations of direct adaptive control system with parametric adjustment?
4. What restrictions are imposed on the $\mathbf{Q}$ matrix in Lyapunov equation?
5. How do adaptive gains values (Γ_A and γ_b) affect the adaptation process?
6. What formulas are used in order to calculate steady values of adaptive coefficients $\mathbf{k}_A$ and k_b ?
7. What is the formula for control action for full state feedback control?
8. Describe the way of calculating full state feedback gain using pole placement method.
9. What does it mean that system is observable?
10. What state variables are used in linearized mathematical model of a double-mass elastic electromechanical system?
11. Which Matlab function can be used in order to solve Lyapunov equation?
12. How does presence of full state feedback controller affect behavior of direct adaptive control system with parametric adjustment?
13. What are the main differences of genetic algorithms from gradient search methods?
14. How is the initial population of genetic algorithm formed?
15. What is a fitness function of genetic algorithm?
16. How can a mutation genetic operator be implemented?
17. How can one set a nonlinear constraints for genetic algorithm in Matlab?
18. What kind of fitness function can be used in order to find $\mathbf{P}$ matrix for direct adaptive control system with parametric adjustment?
19. Why is fixed-step integration method used in genetic algorithm searching for $\mathbf{P}$ matrix for direct adaptive control system with parametric adjustment?
20. What system state space representation is used in Lüders-Narendra adaptive observer and identifier?

21. Describe the way of calculating Lüders-Narendra adaptive observer and identifier.
22. How many auxiliary signals are introduced in Lüders-Narendra adaptive observer and identifier for the system of n -th order?
23. How many unknown system parameters are identified by Lüders-Narendra adaptive observer and identifier for the system of n -th order?
24. Describe the way of calculating sliding mode observer.
25. How is system transformation matrix for sliding mode observer calculated?

REFERENCES

1. An-chyau Huang, Ming-chih Chien. Adaptive Control of Robot Manipulators: A Unified Regressor-free Approach.
2. Gerd Lüders, Kumpati S. Narendra. An Adaptive Observer and Identifier for a Linear System.
3. Karl Johan Astrom, Bjorn Wittenmark. Adaptive Control.

TABLE OF CONTENTS

1. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS WITH PARAMETRIC ADAPTATION FOR	3
FIRST-ORDER PLANTS	3
2. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS WITH PARAMETRIC ADAPTATION FOR	10
SECOND-ORDER PLANTS	10
3. SYNTHESIS AND INVESTIGATION OF SYSTEMS WITH FULL STATE FEEDBACK CONTROL AND STATE OBSERVER.....	16
4. SYNTHESIS AND INVESTIGATION OF ADAPTIVE CONTROL SYSTEMS FOR DOUBLE-MASS ELASTIC ELECTROMECHANICAL SYSTEM	22
5. GENETIC ALGORITHMS.....	29
6. ADAPTIVE CONTROL SYSTEM TUNING BASED ON GENETIC ALGORITHMS.....	37
7. LÜDERS-NARENDRA ADAPTIVE OBSERVER AND.....	46
IDENTIFIER.....	46
8. SLIDING MODE STATE OBSERVER	51
TEST QUESTIONS	60
REFERENCES.....	62
TABLE OF CONTENTS	63